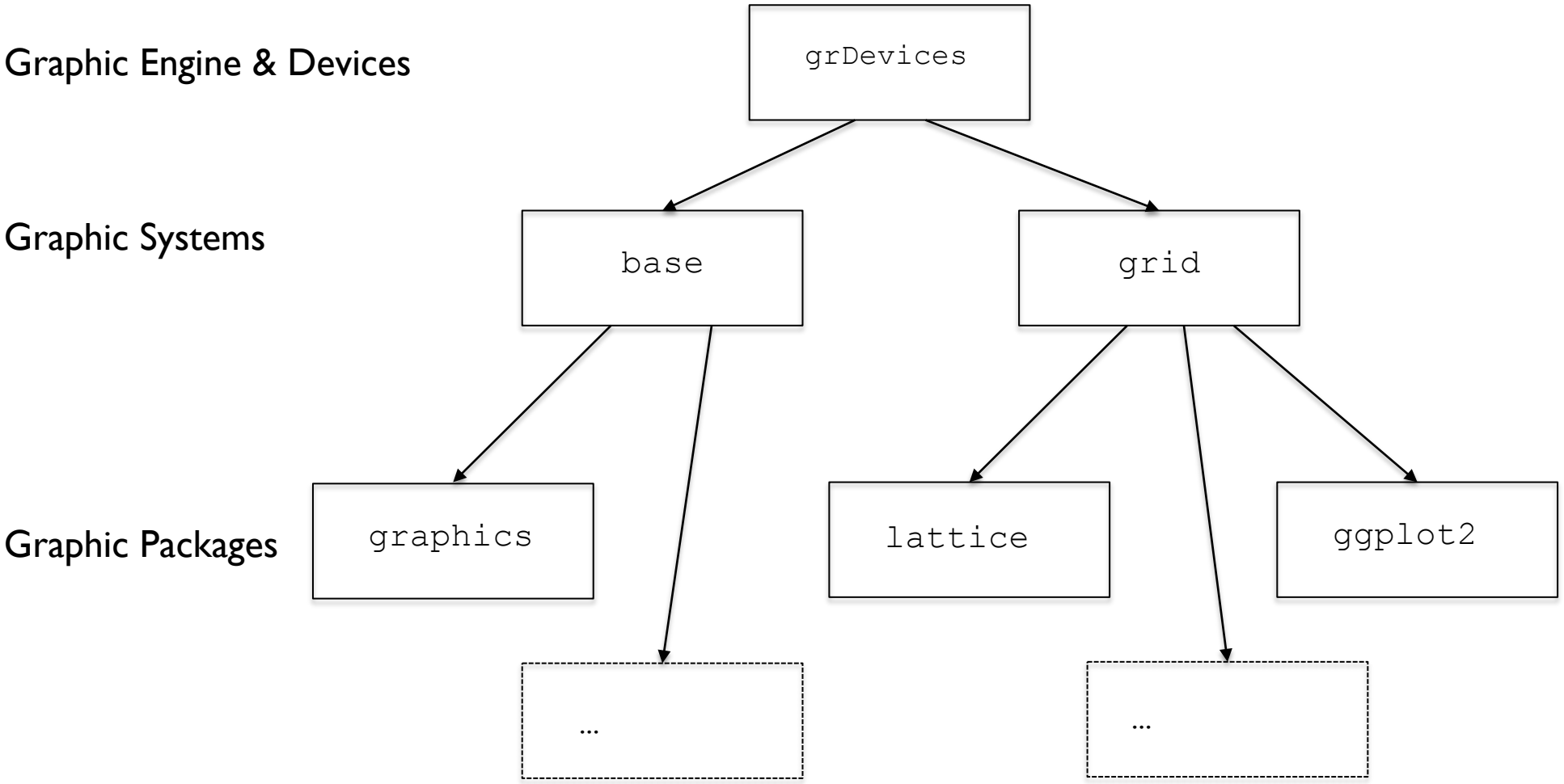
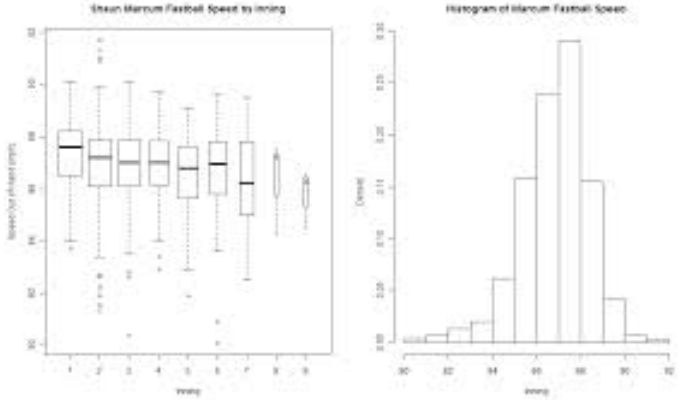
How to do it in R

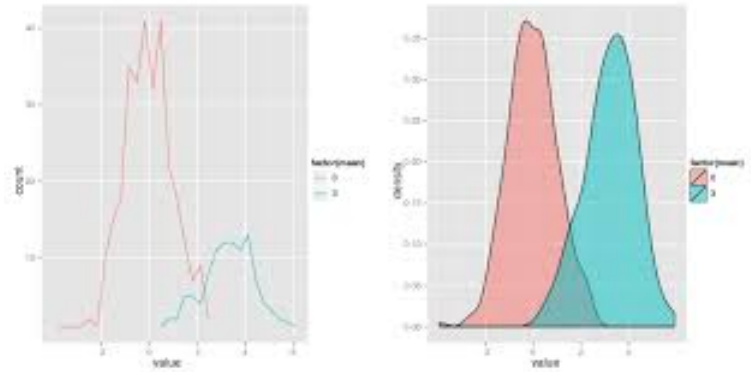
R Graphics


Graphic Engine & Devices
Graphic Systems
Graphic Packages
grDevices
base
grid
graphics
lattice
ggplot2
...
...

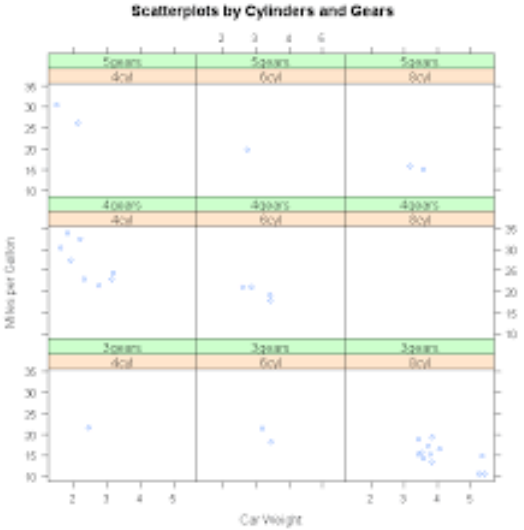
R Graphic Packages

base graphics





lattice



ggplot2

Graphics

High-level commands: create a plot of a particular format

Low-level commands: add elements to a plot (points, lines, text, etc.)

Arguments and parameters: define and fine-tune various elements in a plot (color, size, etc.)

```
parameters(arguments)

highlevelcommand(data, arguments)

lowlevelcommand(data, arguments)

lowlevelcommand(data, arguments)

lowlevelcommand(data, arguments)
```

Before Graphing

Set Working Directory

```
setwd("/YOURDIRECTORY/")
```

Getting Data into R

```
library(foreign)
data <- read.csv("gapminder.csv", sep=";")
```

Before Graphing

Quick Look at Data Set

```
head(data)
```

```
      country continent year lifeExp      pop gdpPercap
1 Afghanistan      Asia 1952  28.801  8425333  779.4453
2 Afghanistan      Asia 1957  30.332  9240934  820.8530
3 Afghanistan      Asia 1962  31.997 10267083  853.1007
4 Afghanistan      Asia 1967  34.020 11537966  836.1971
5 Afghanistan      Asia 1972  36.088 13079460  739.9811
6 Afghanistan      Asia 1977  38.438 14880372  786.1134
```

Before Graphing

Look at Single Variables

```
class(data$lifeExp)
[1] "numeric"

class(data$continent)
[1] "factor"

summary(data$lifeExp)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  23.60   48.20   60.71   59.47   70.85   82.60

summary(data$continent)
  Africa Americas     Asia   Europe  Oceania
     624      300      396      360       24
```

Before Graphing

Creating a Simple Table

```
table(data$continent)
```

```
  Africa Americas     Asia   Europe  Oceania
     624      300      396      360       24
```

```
my.tab <- table(data$continent)
prop.table(my.tab)
```

```
    Africa   Americas       Asia     Europe    Oceania
0.36619718 0.17605634 0.23239437 0.21126761 0.01408451
```

Before Graphing

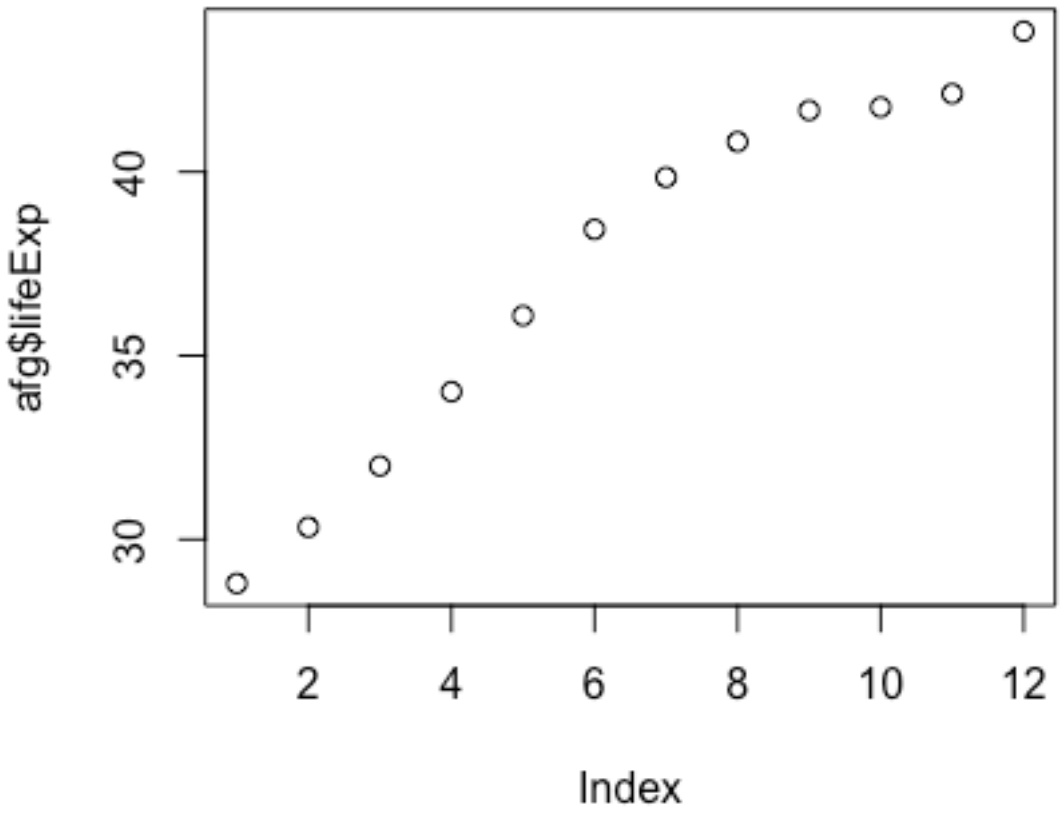
Subsetting the Data Set

```
afg <- data[data$country=="Afghanistan",]
```

Creating Plots Using High-level Functions

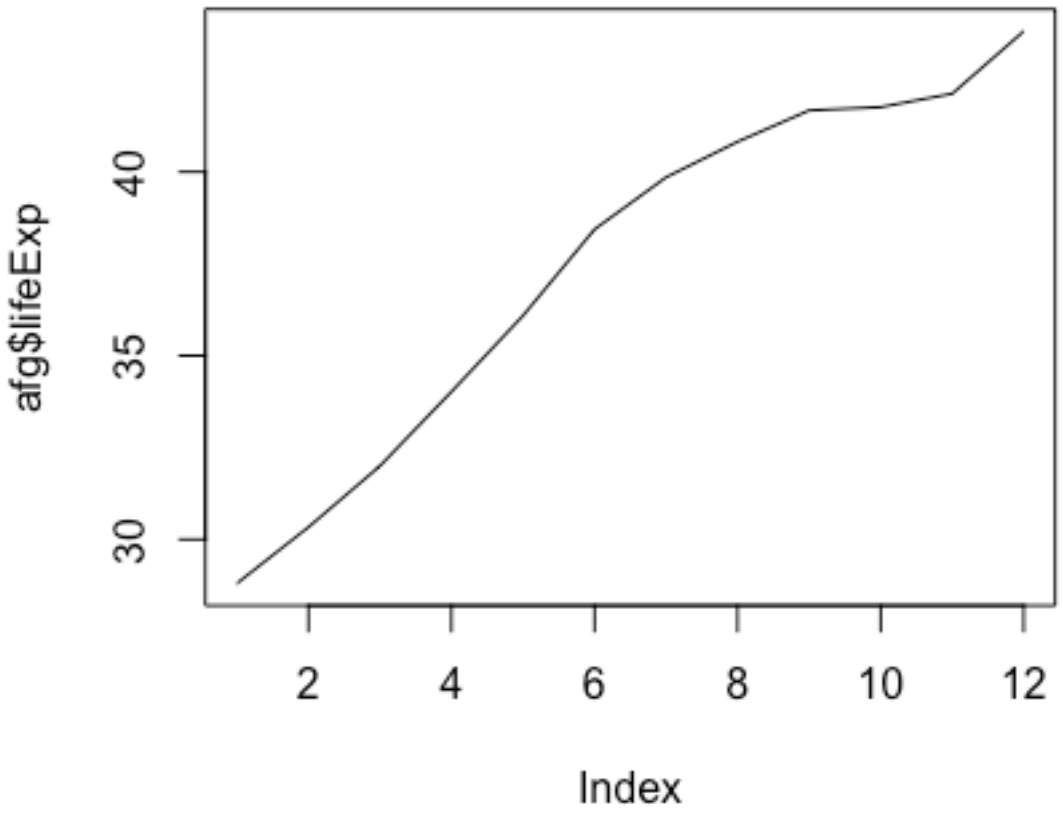
Some High-level Functions

```
plot(afg$lifeExp)
```



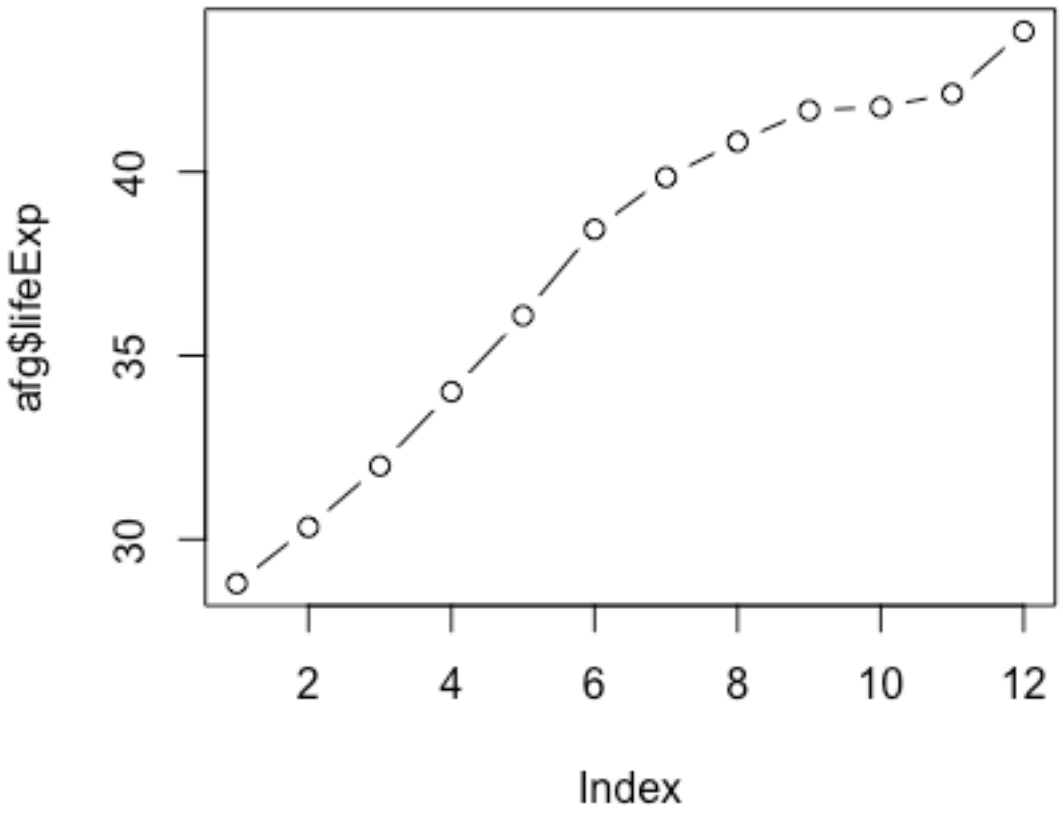
Some High-level Functions

```
plot(afg$lifeExp, type="l")
```



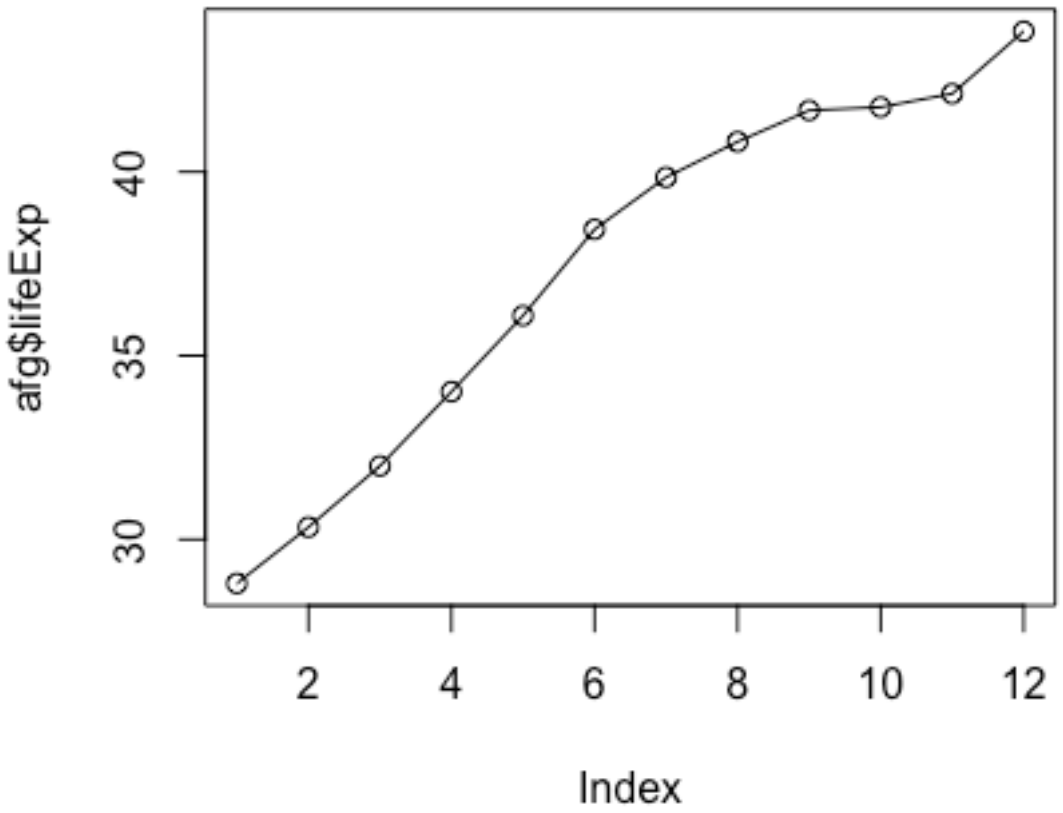
Some High-level Functions

```
plot(afg$lifeExp, type="b")
```



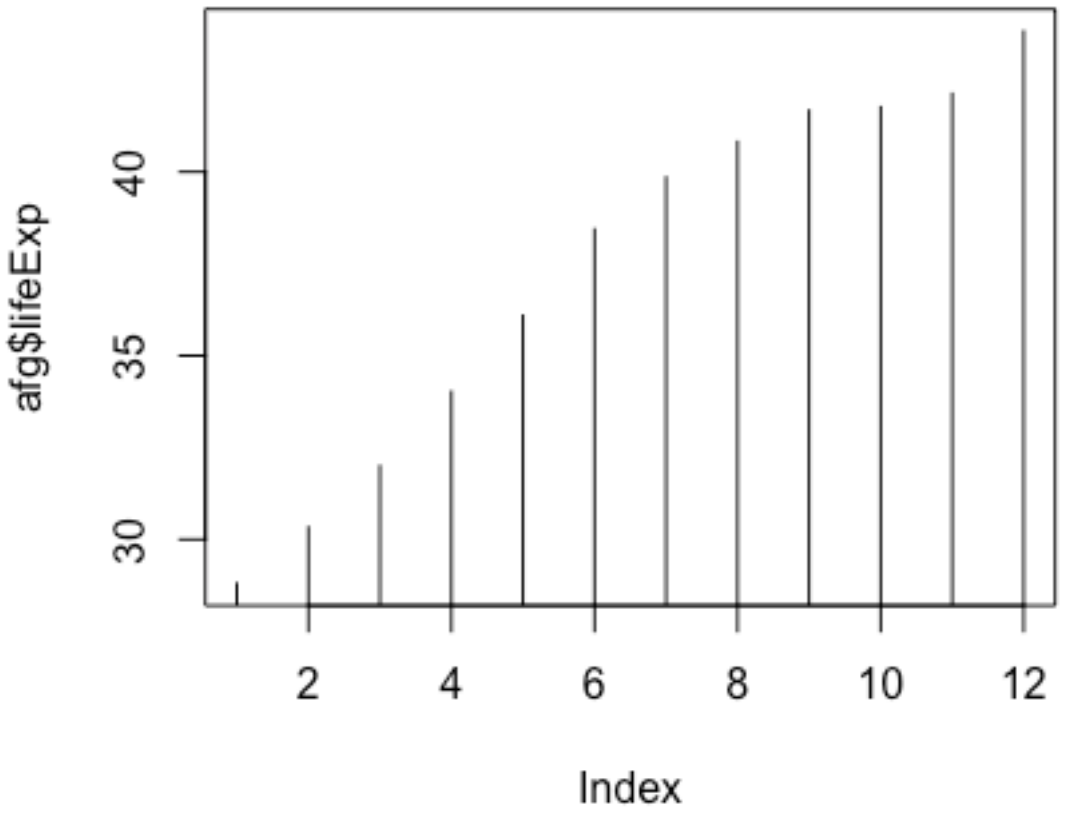
Some High-level Functions

```
plot(afg$lifeExp, type="o")
```



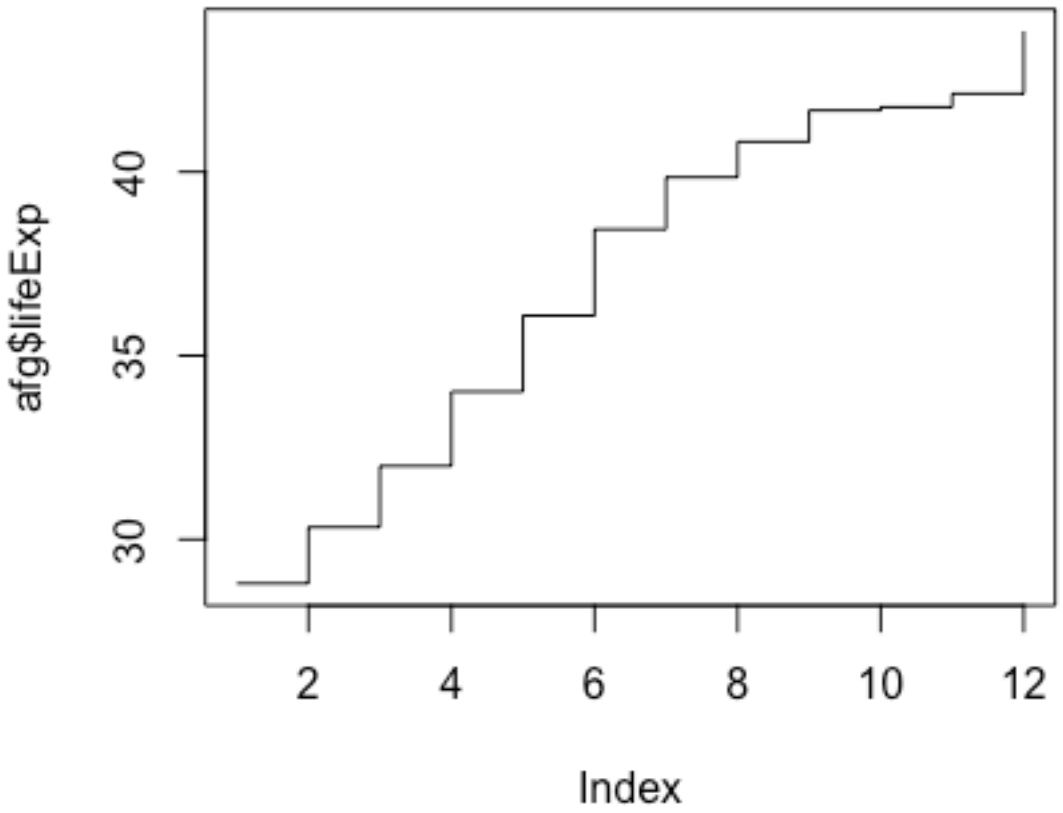
Some High-level Functions

```
plot(afg$lifeExp, type="h")
```



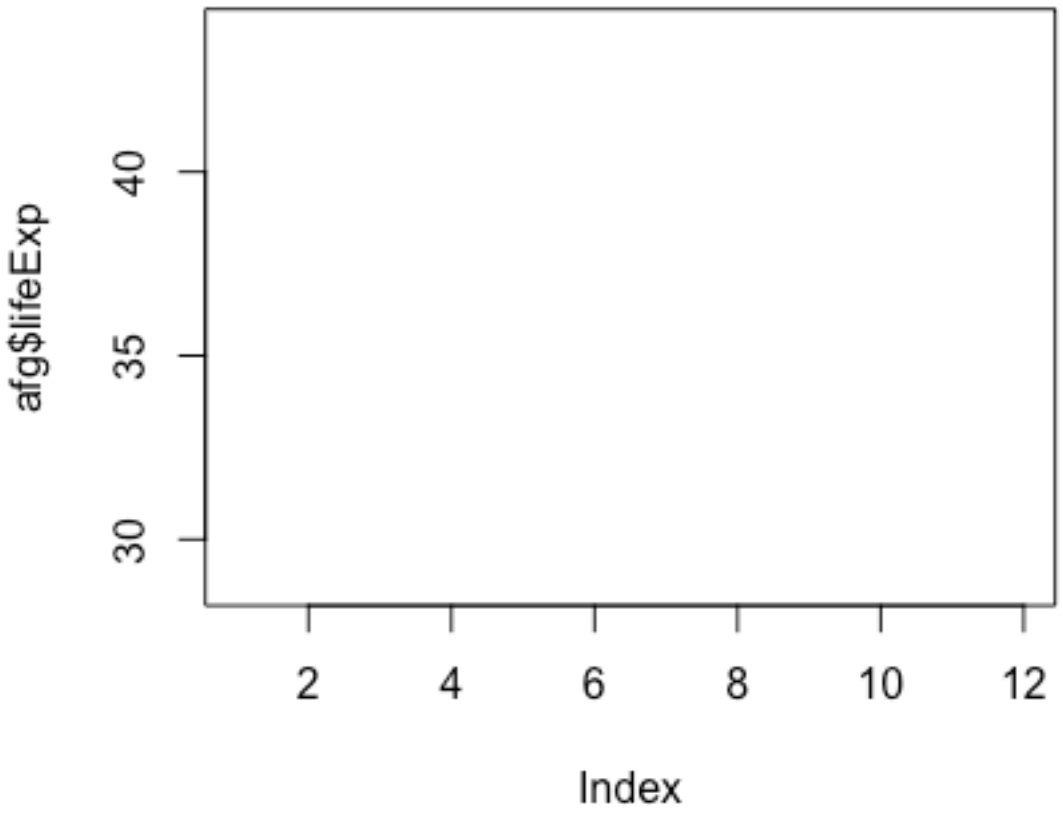
Some High-level Functions

```
plot(afg$lifeExp, type="s")
```



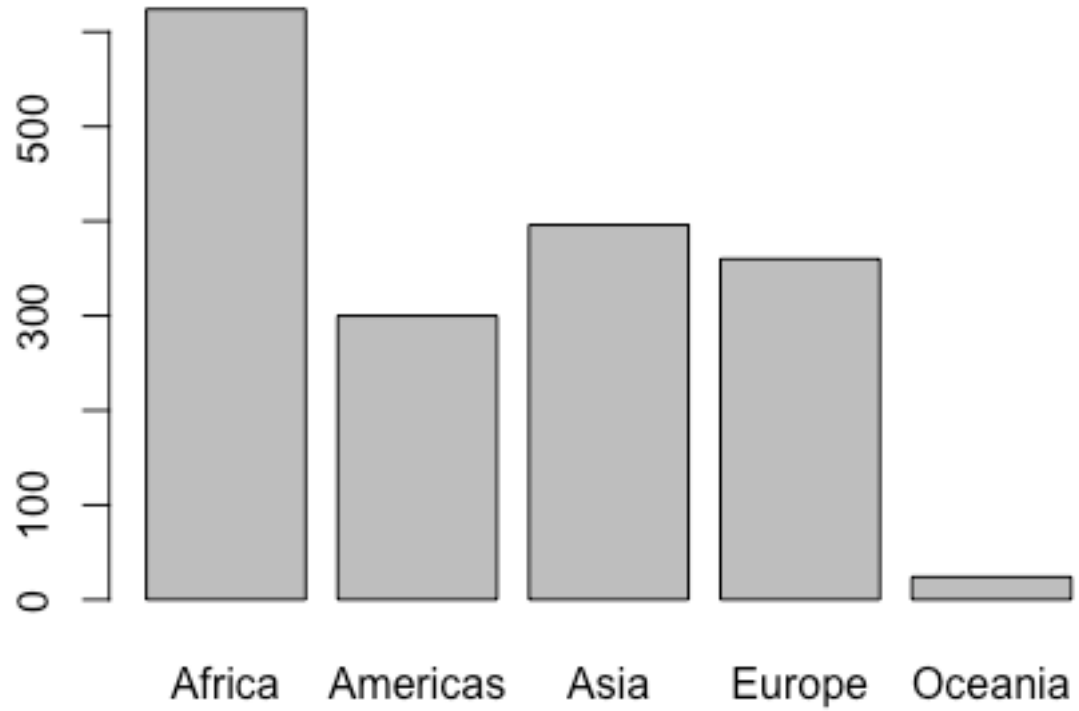
Some High-level Functions

```
plot(afg$lifeExp, type="n")
```



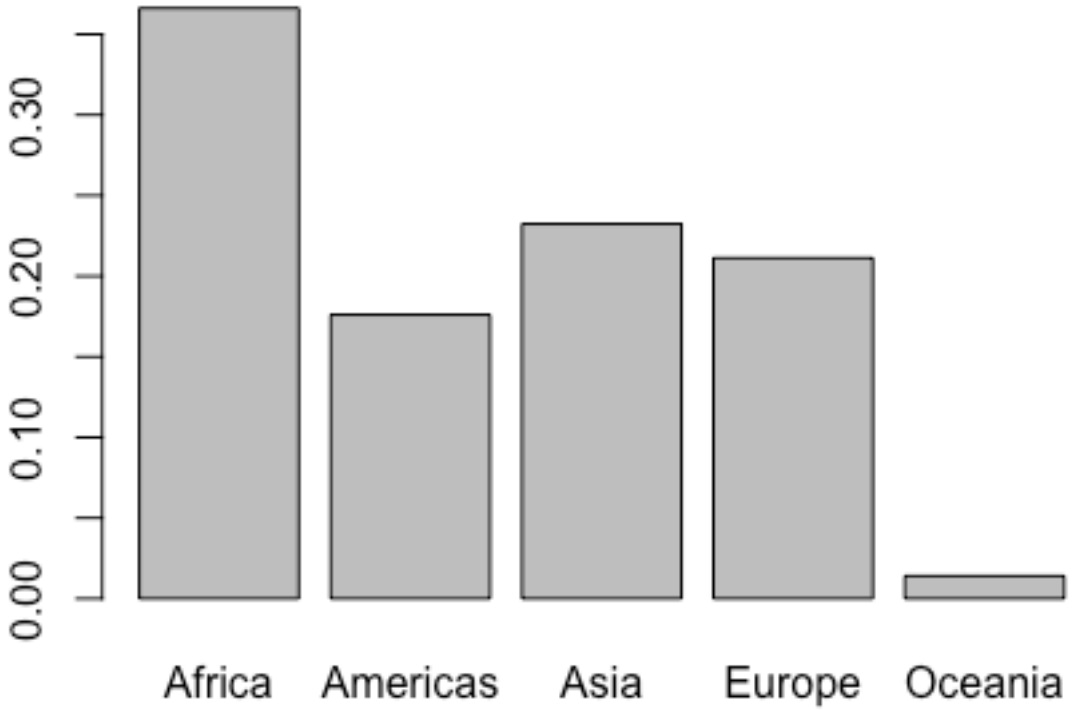
Some High-level Functions

```
plot(data$continent)
```



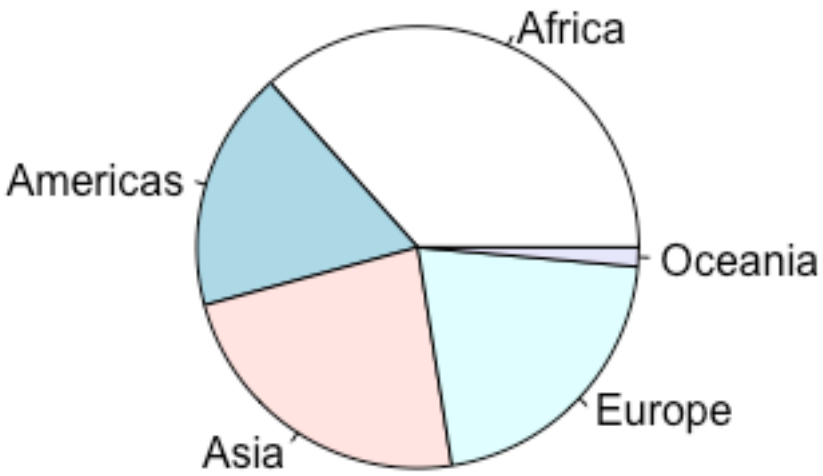
Some High-level Functions

```
my.tab <- prop.table(table(data$continent))

barplot(my.tab)
```



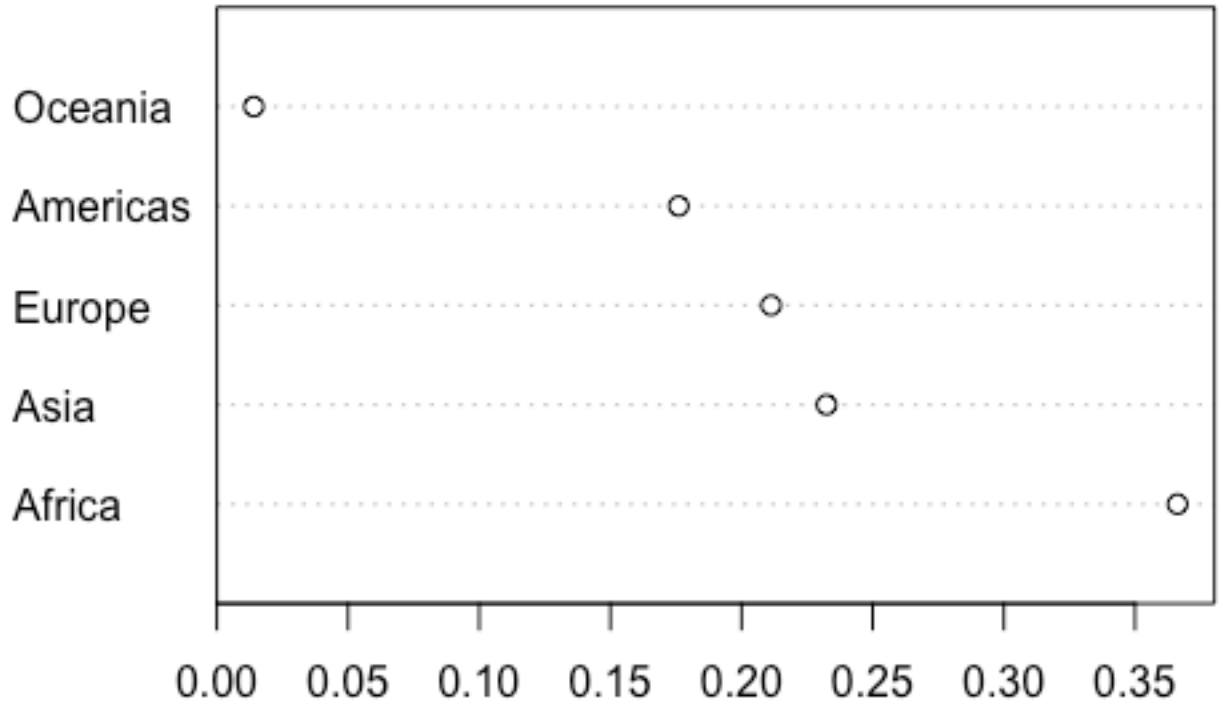
Some High-level Functions

```
pie(my.tab)
```



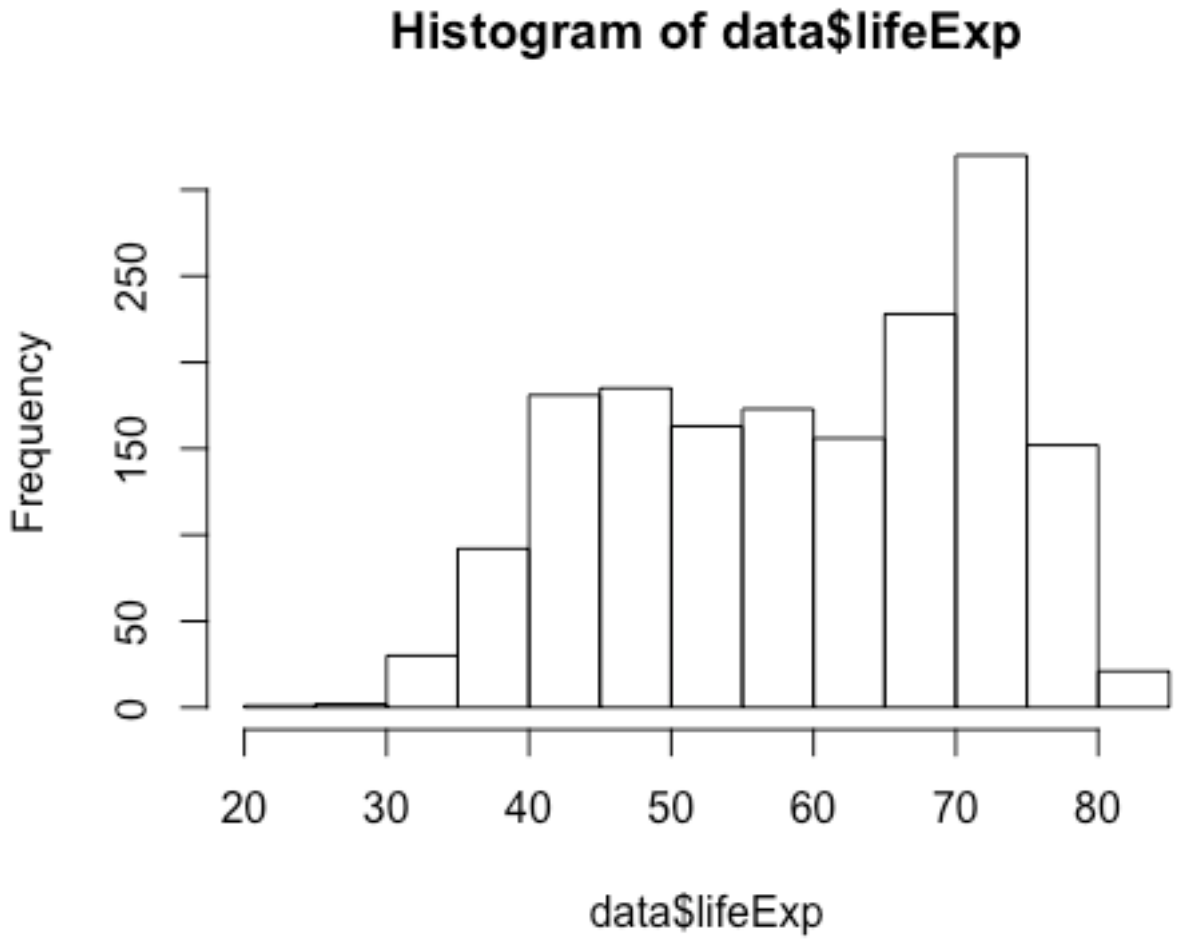
Some High-level Functions

```
dotchart(my.tab)
```



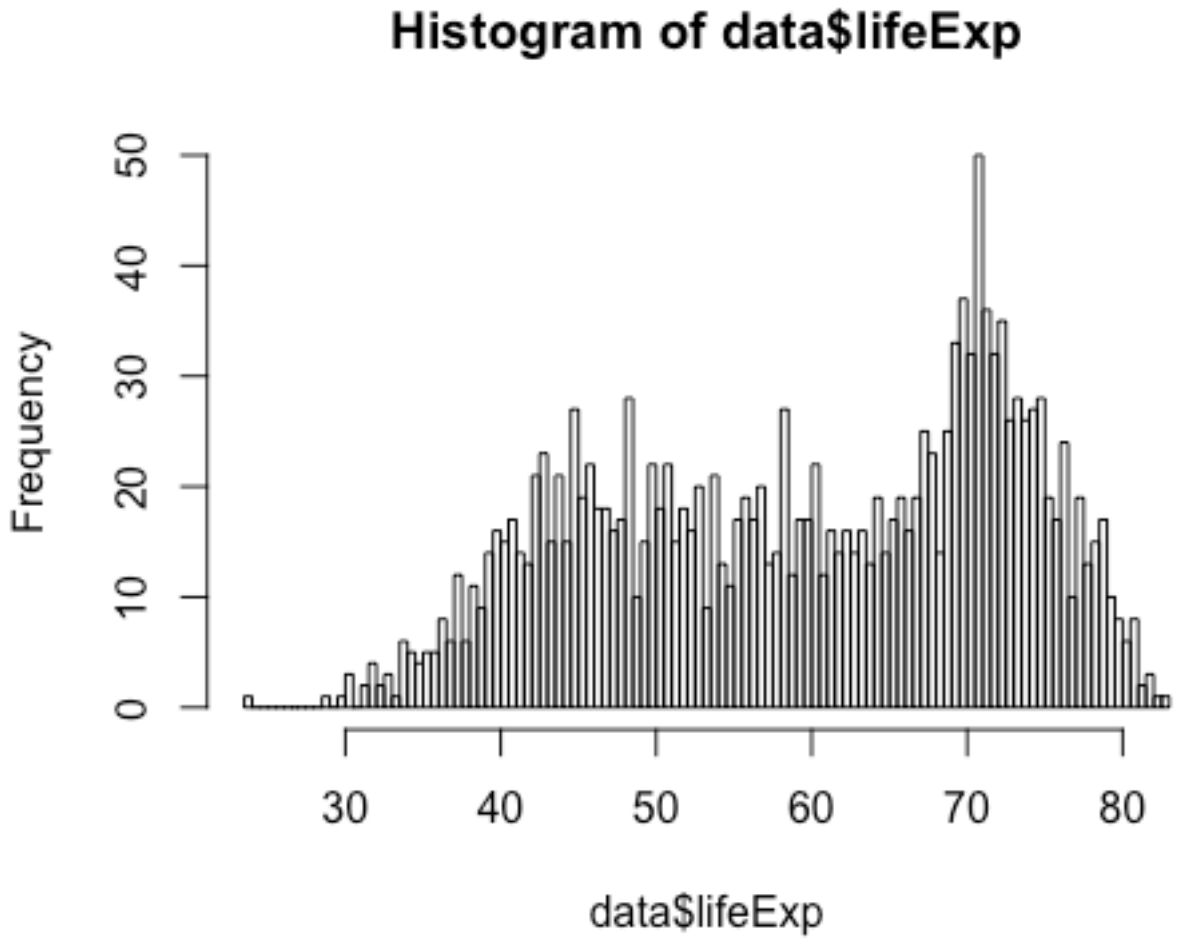
Some High-level Functions

```
hist(data$lifeExp)
```



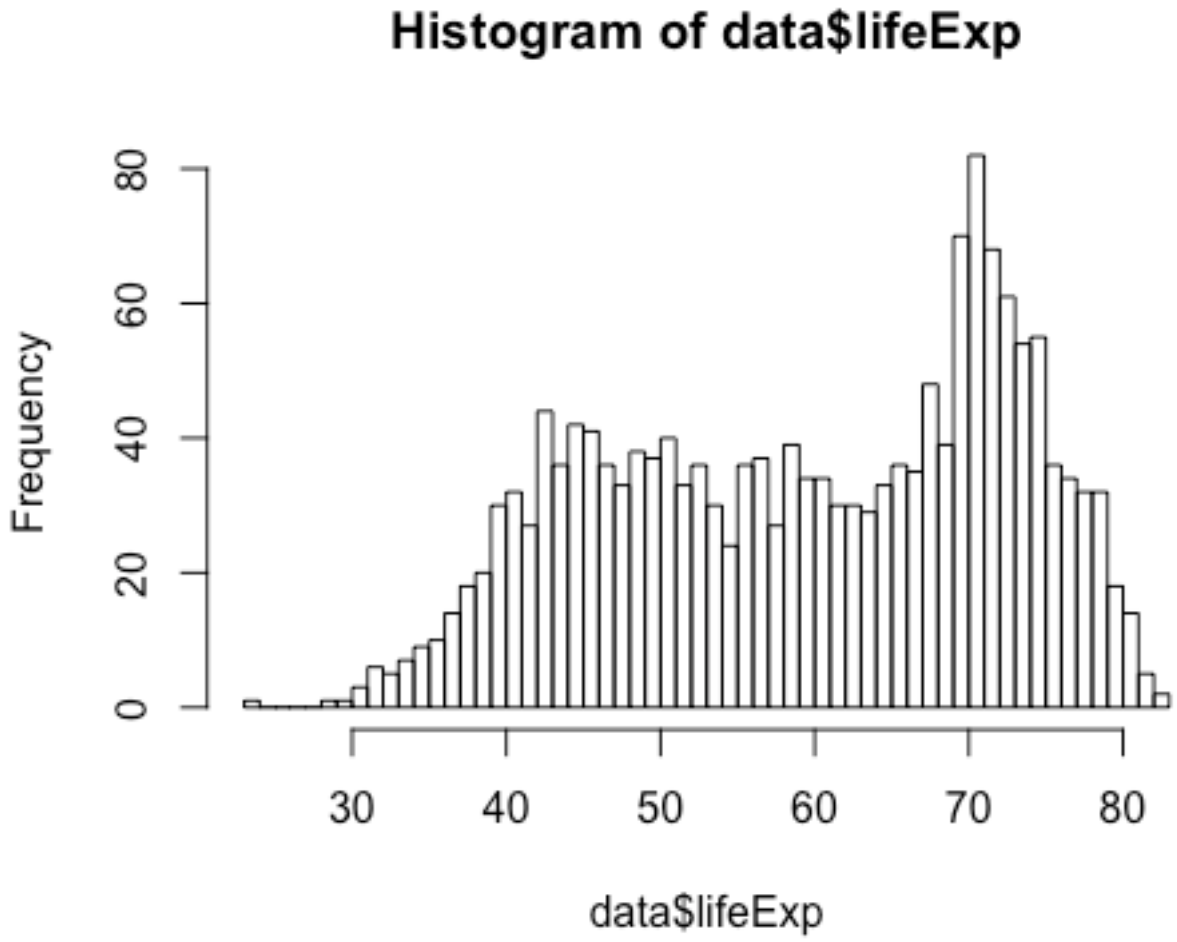
Some High-level Functions

```
hist(data$lifeExp, breaks=100)
```



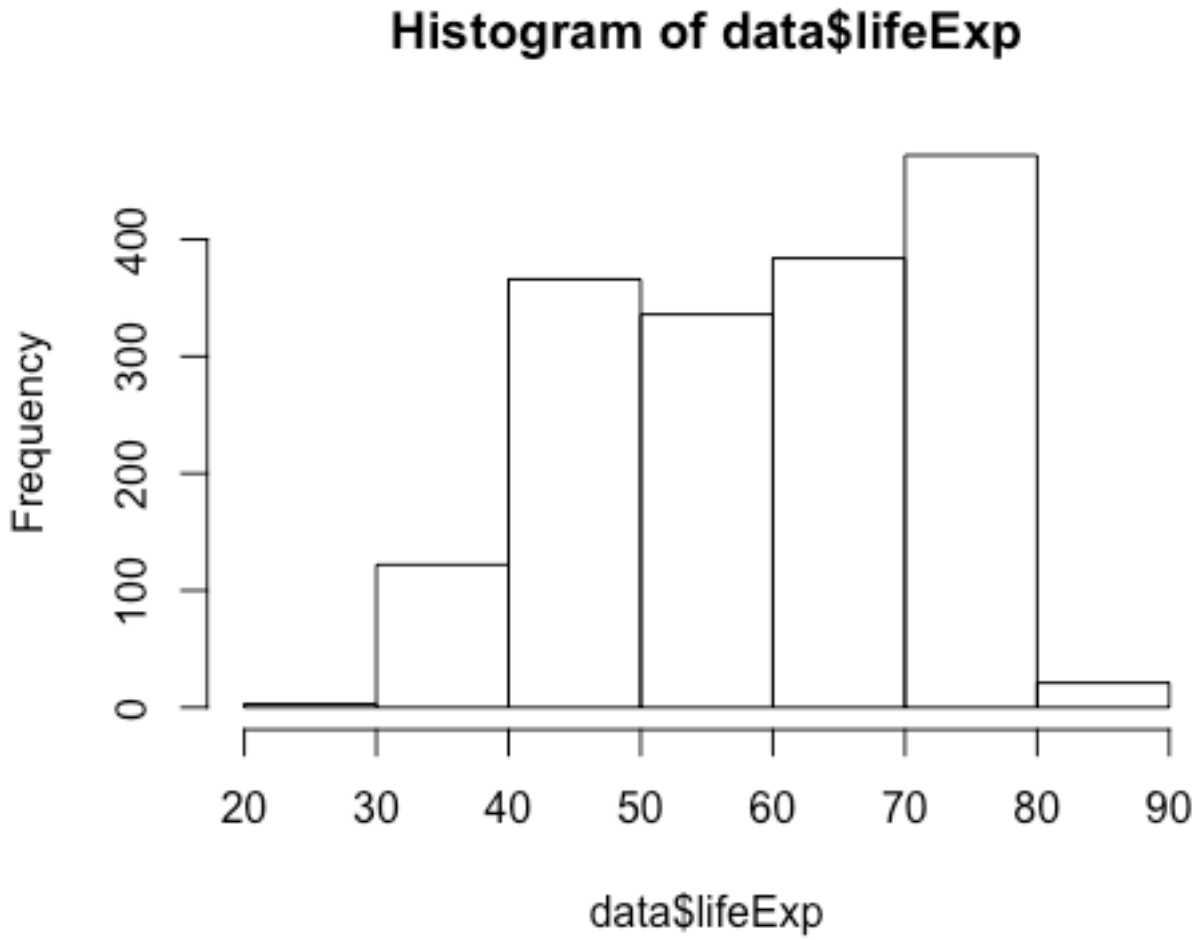
Some High-level Functions

```
hist(data$lifeExp, breaks=50)
```



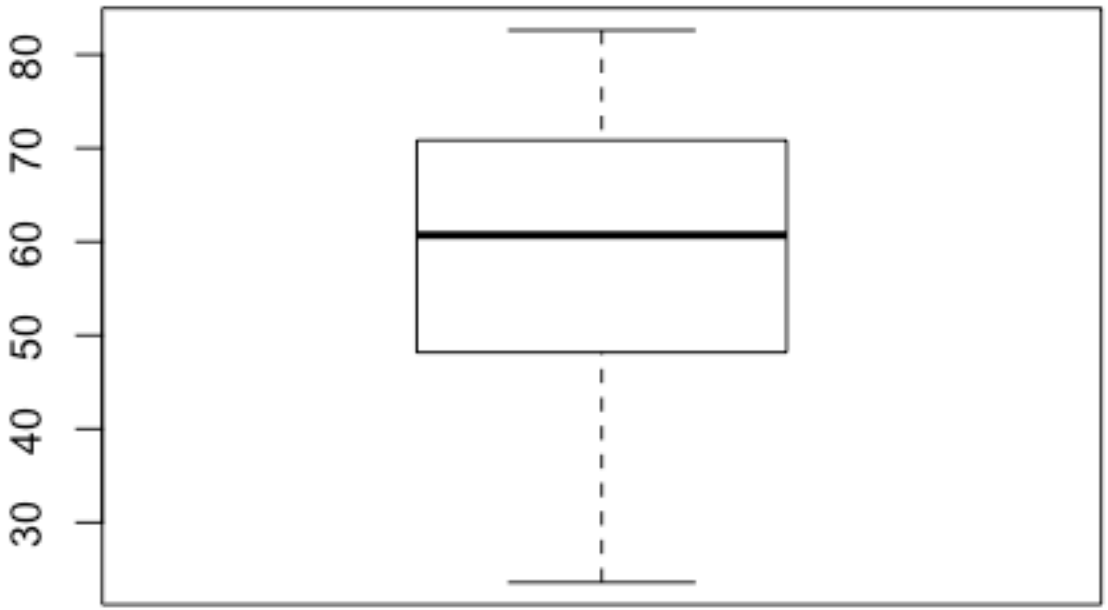
Some High-level Functions

```
hist(data$lifeExp, breaks=5)
```



Some High-level Functions

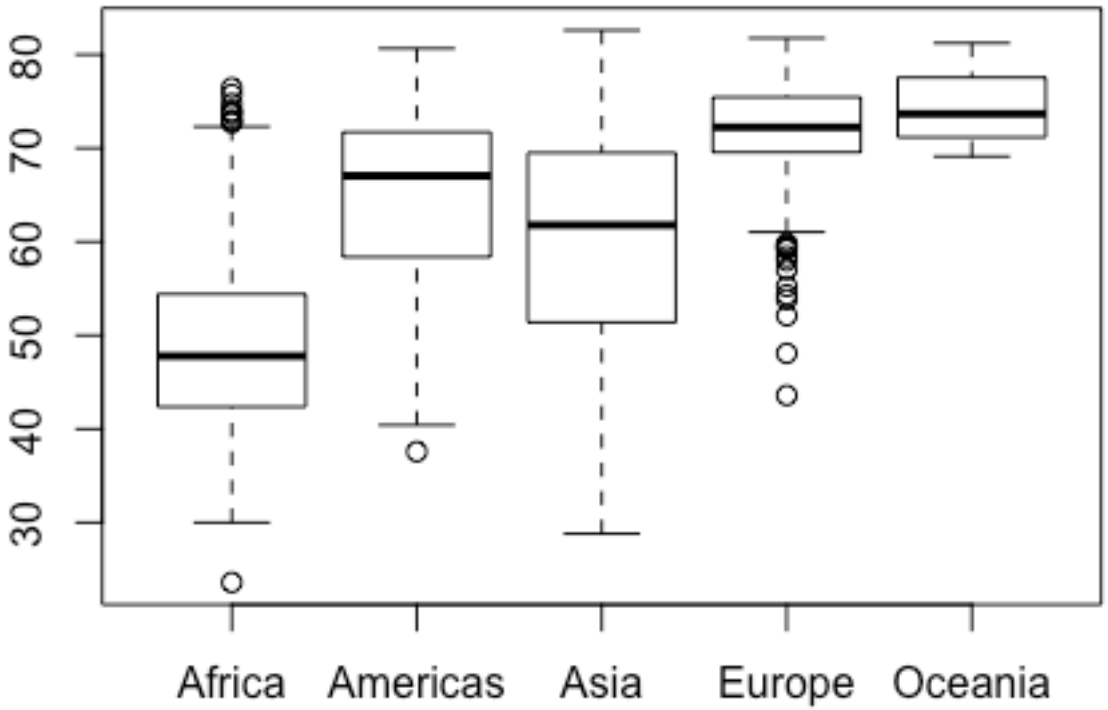
```
boxplot(data$lifeExp)
```



Adding a Second Variable

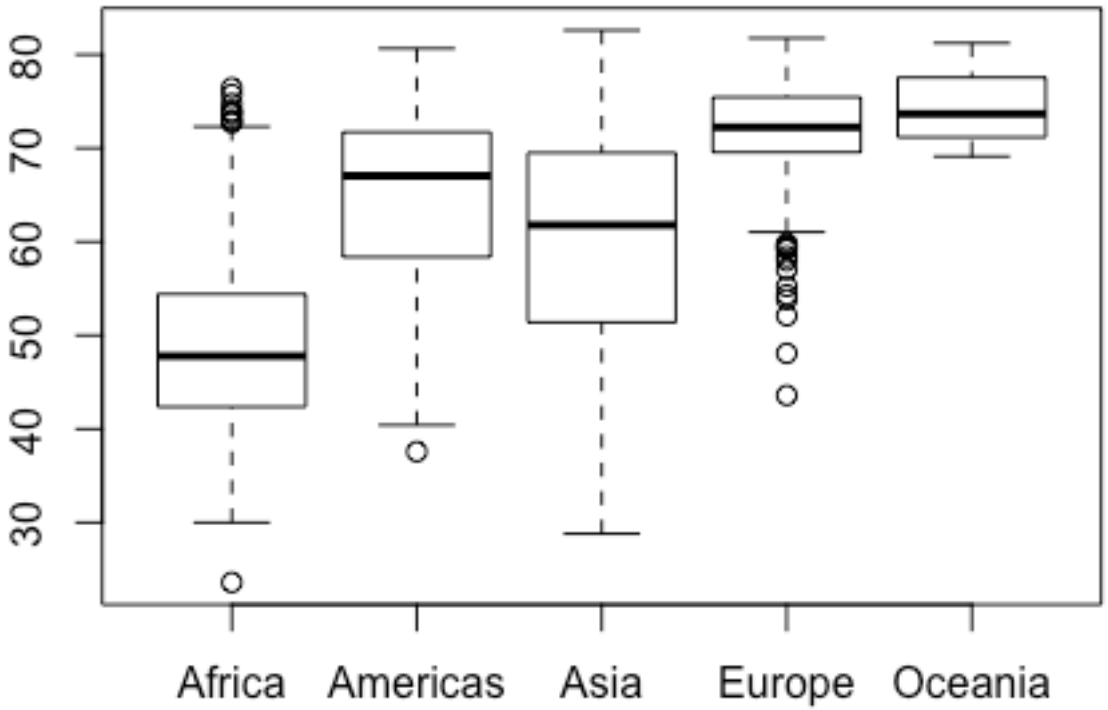
Adding a Second Variable

```
boxplot(data$lifeExp ~ data$continent)
```



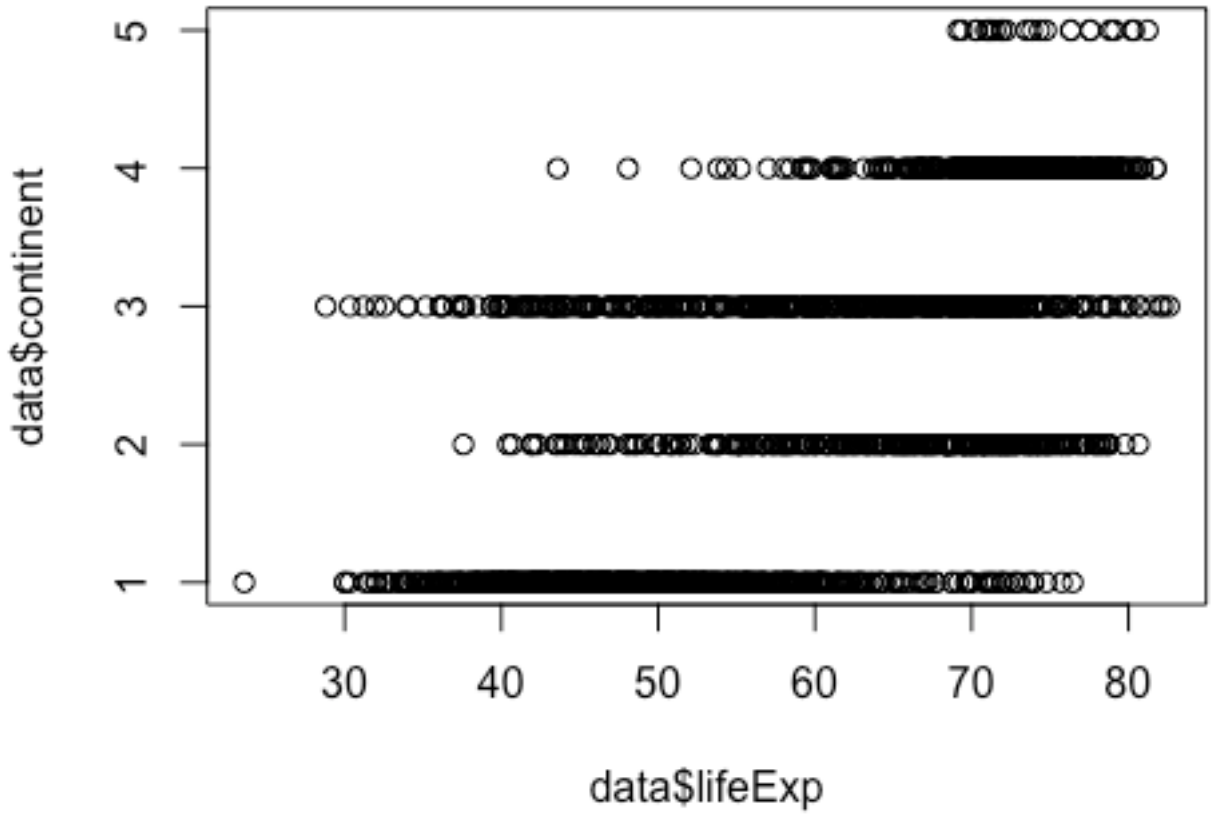
Adding a Second Variable

```
plot(data$continent, data$lifeExp)
```



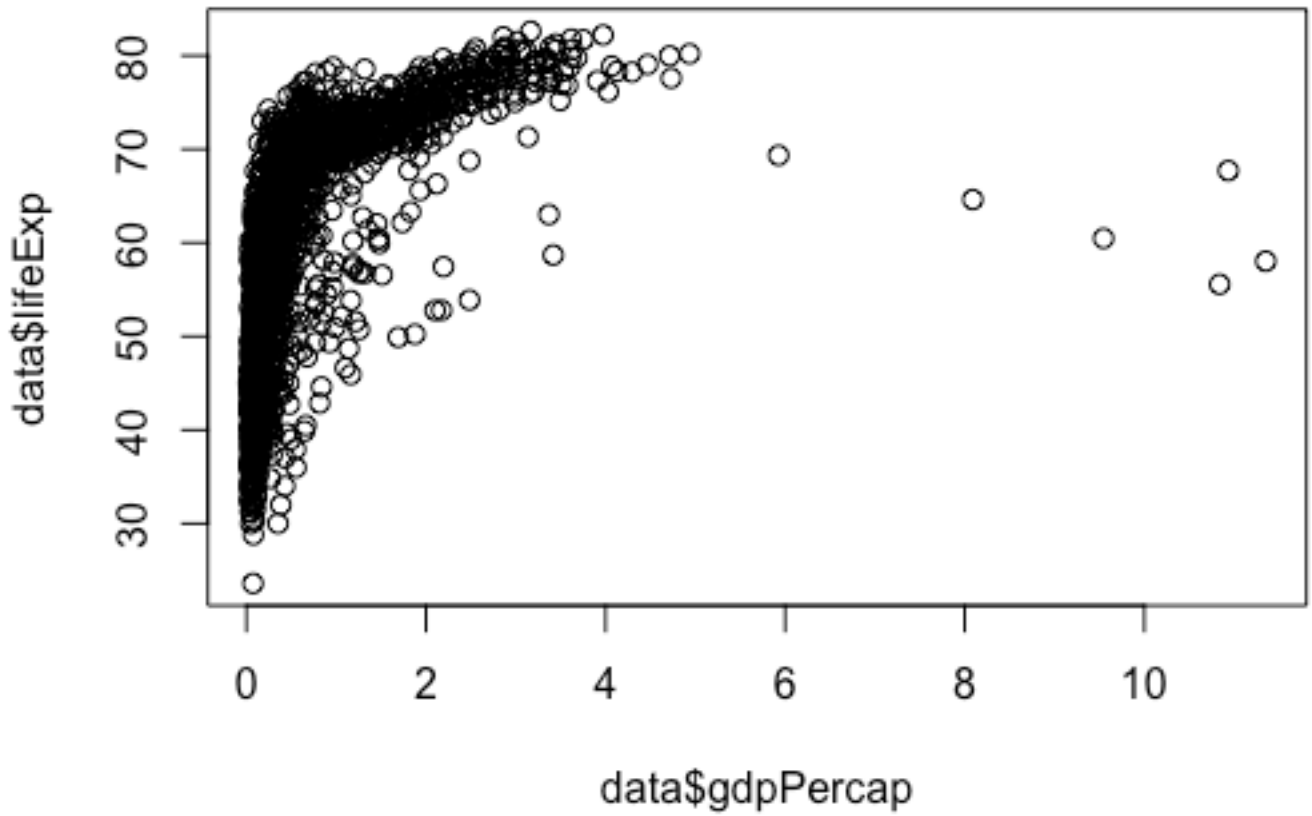
Adding a Second Variable

```
plot(data$lifeExp, data$continent)
```



Adding a Second Variable

```
data$gdpPercap <- data$gdpPercap/10000

plot(data$gdpPercap, data$lifeExp)
```

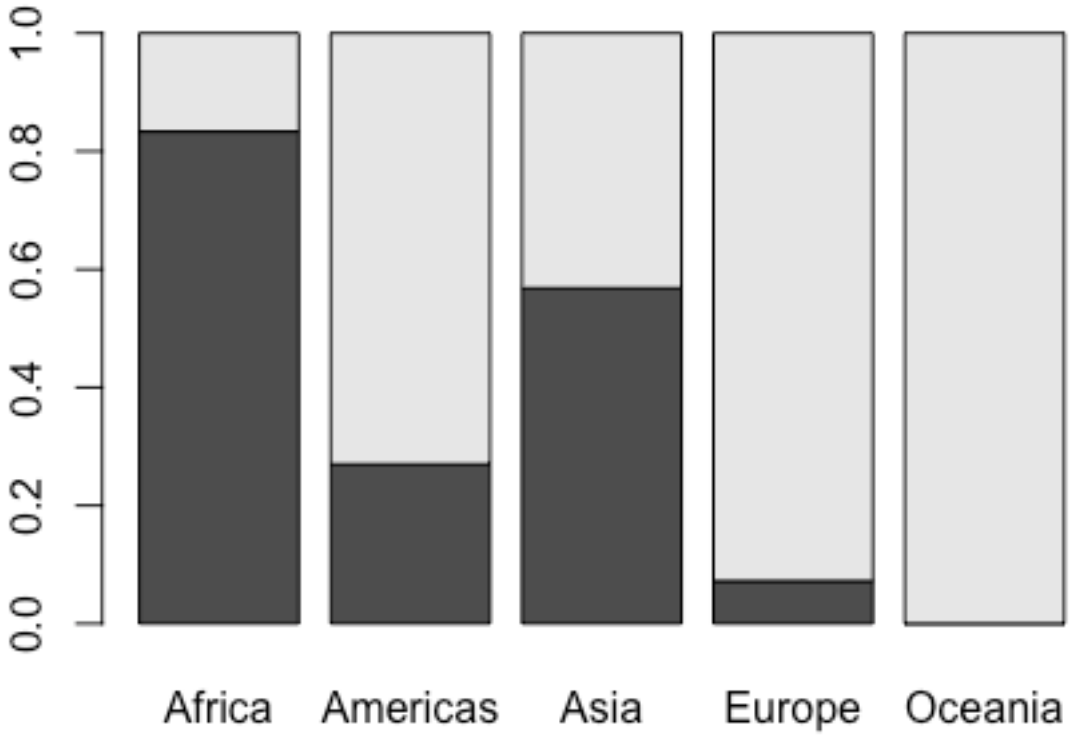


Adding a Second Variable

```
data$rich <- ifelse(data$gdpPerc > median(data$gdpPercap),
"rich", "poor")

data$rich <- as.factor(data$rich)


my.tab <- prop.table(table(data$rich, data$continent), 2)


> my.tab

            Africa   Americas       Asia     Europe    Oceania
  poor 0.83333333 0.27000000 0.56818182 0.07222222 0.00000000
  rich 0.16666667 0.73000000 0.43181818 0.92777778 1.00000000
```

Adding a Second Variable

```
barplot(my.tab)
```



Ordering a Categorical Variable

```
ord <- order(my.tab[2,])
```

```
> ord
[1] 1 3 2 4 5
```

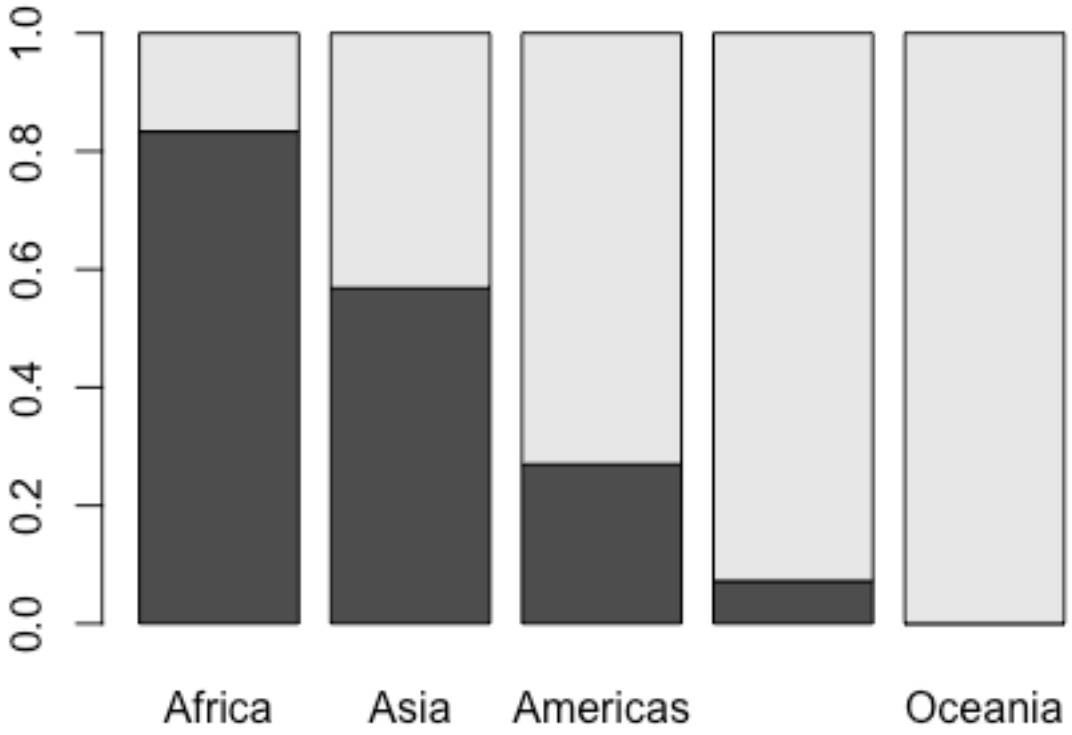
```
my.tab <- my.tab[, ord]
```

```
> my.tab
```

	Africa	Asia	Americas	Europe	Oceania
poor	0.83333333	0.56818182	0.27000000	0.07222222	0.00000000
rich	0.16666667	0.43181818	0.73000000	0.92777778	1.00000000

Ordering a Categorical Variable

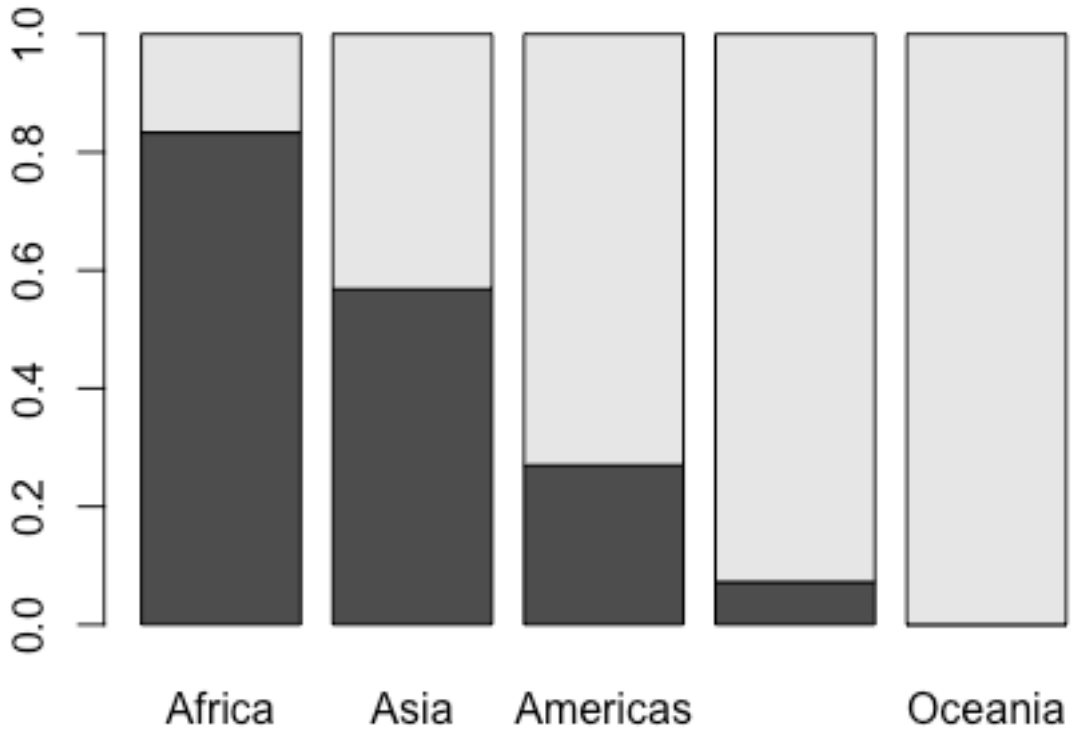
```
barplot(my.tab)
```



Changing Elements of a Graphic

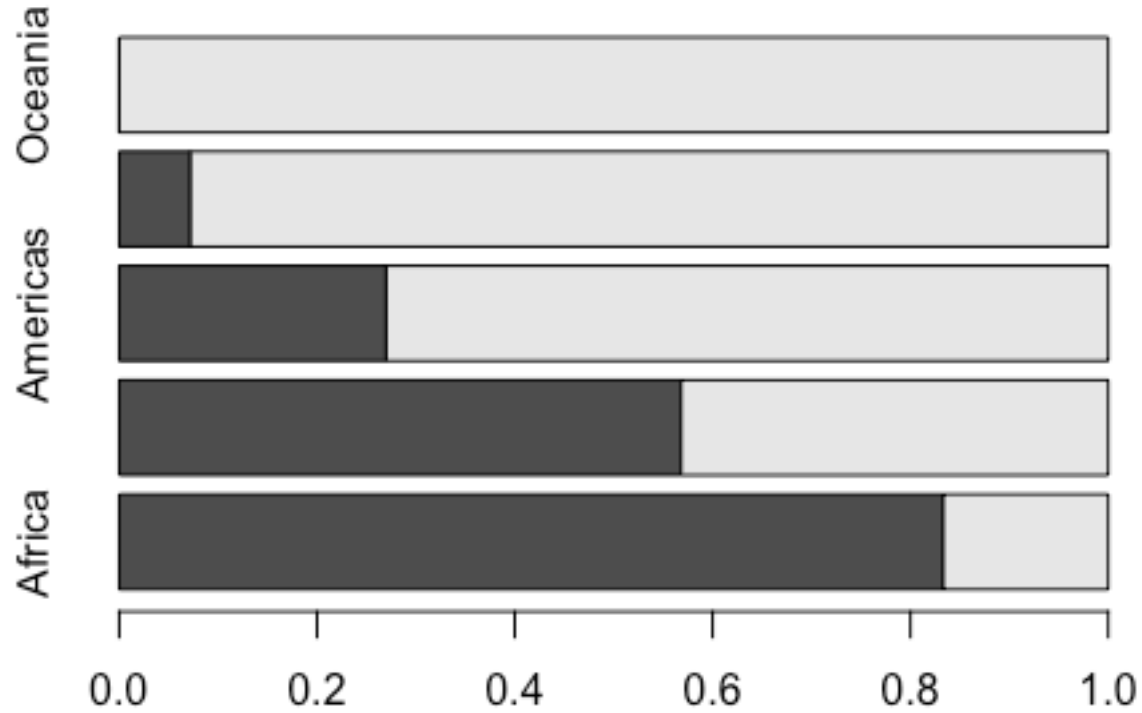
Some Arguments: Graphical Parameters

```
barplot(my.tab)
```



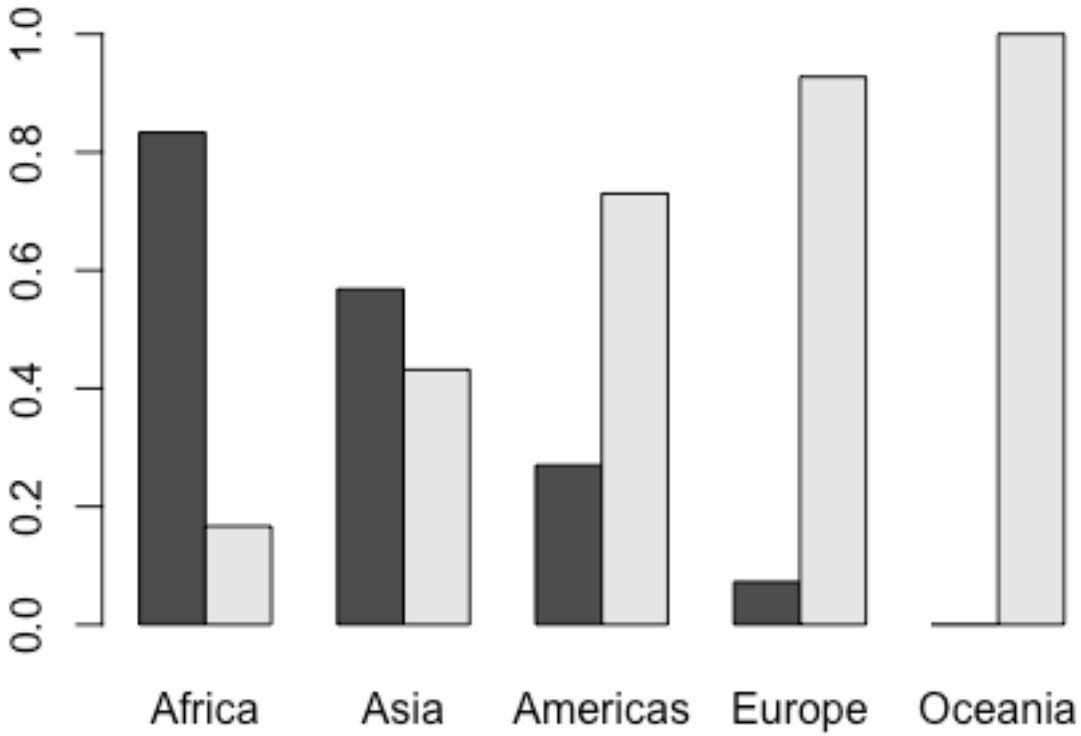
Some Arguments: Graphical Parameters

```
barplot(my.tab, horiz=T)
```



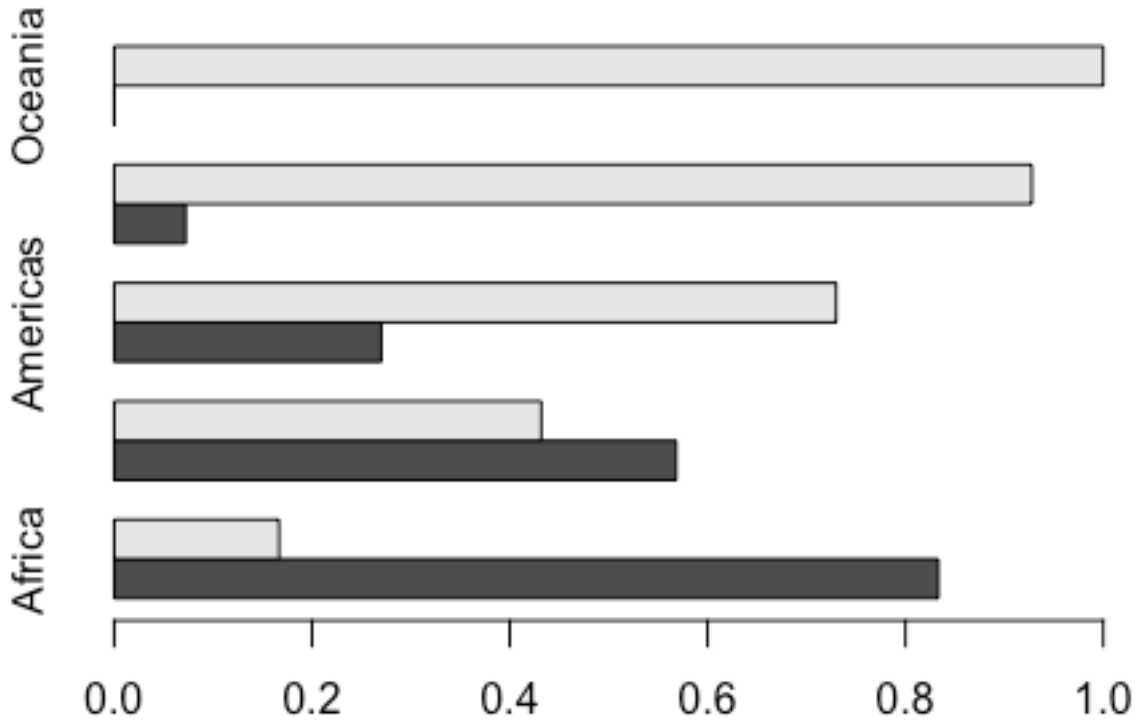
Some Arguments: Graphical Parameters

```
barplot(my.tab, beside=T)
```



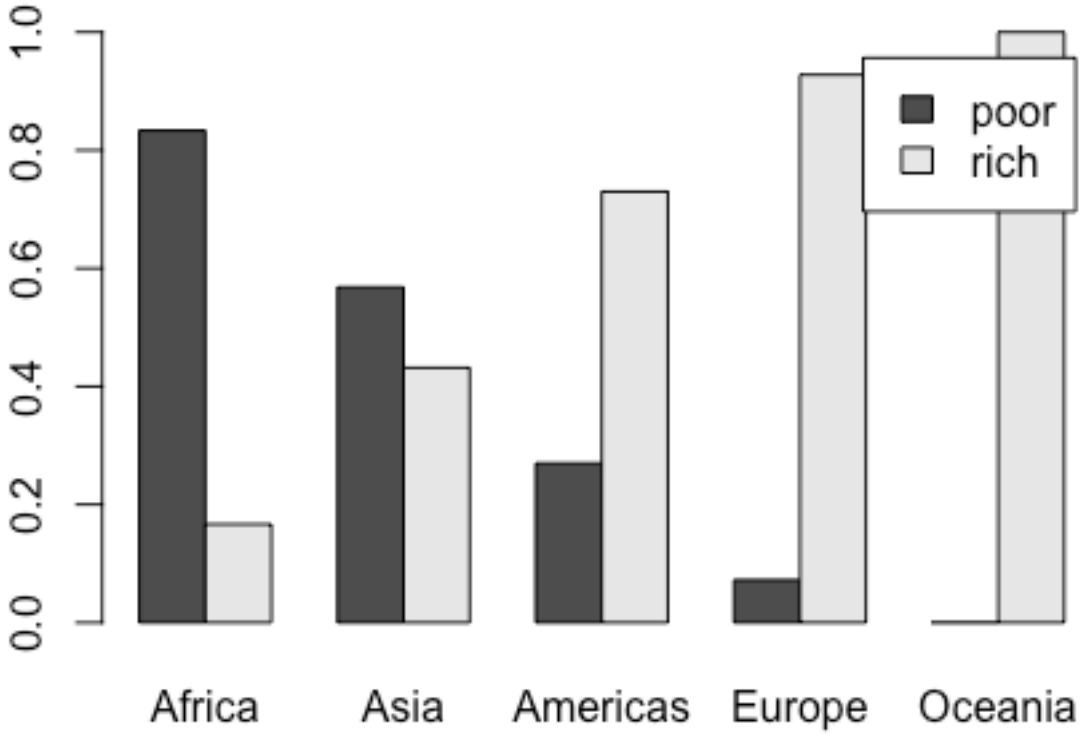
Some Arguments: Graphical Parameters

```
barplot(my.tab, beside=T, horiz=T)
```



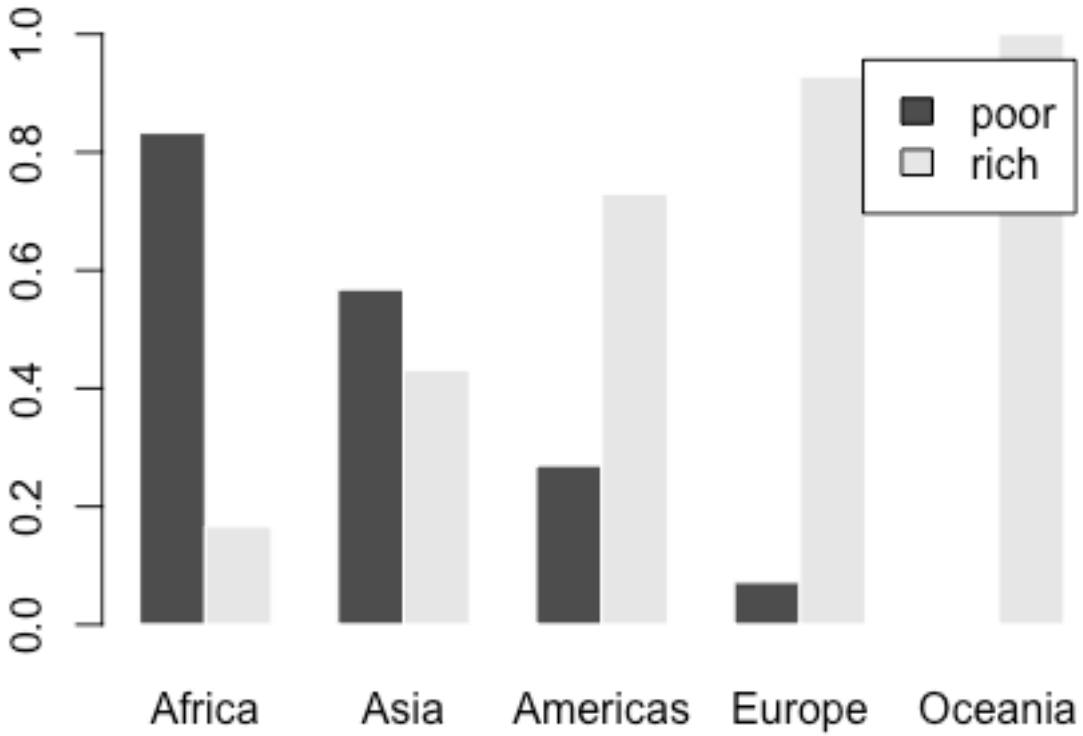
Some Arguments: Graphical Parameters

```
barplot(my.tab, beside=T, legend=T)
```



Some Arguments: Graphical Parameters

```
barplot(my.tab, beside=T, legend=T, border=F)
```



Use Help to Find More Arguments

```
?barplot()
```

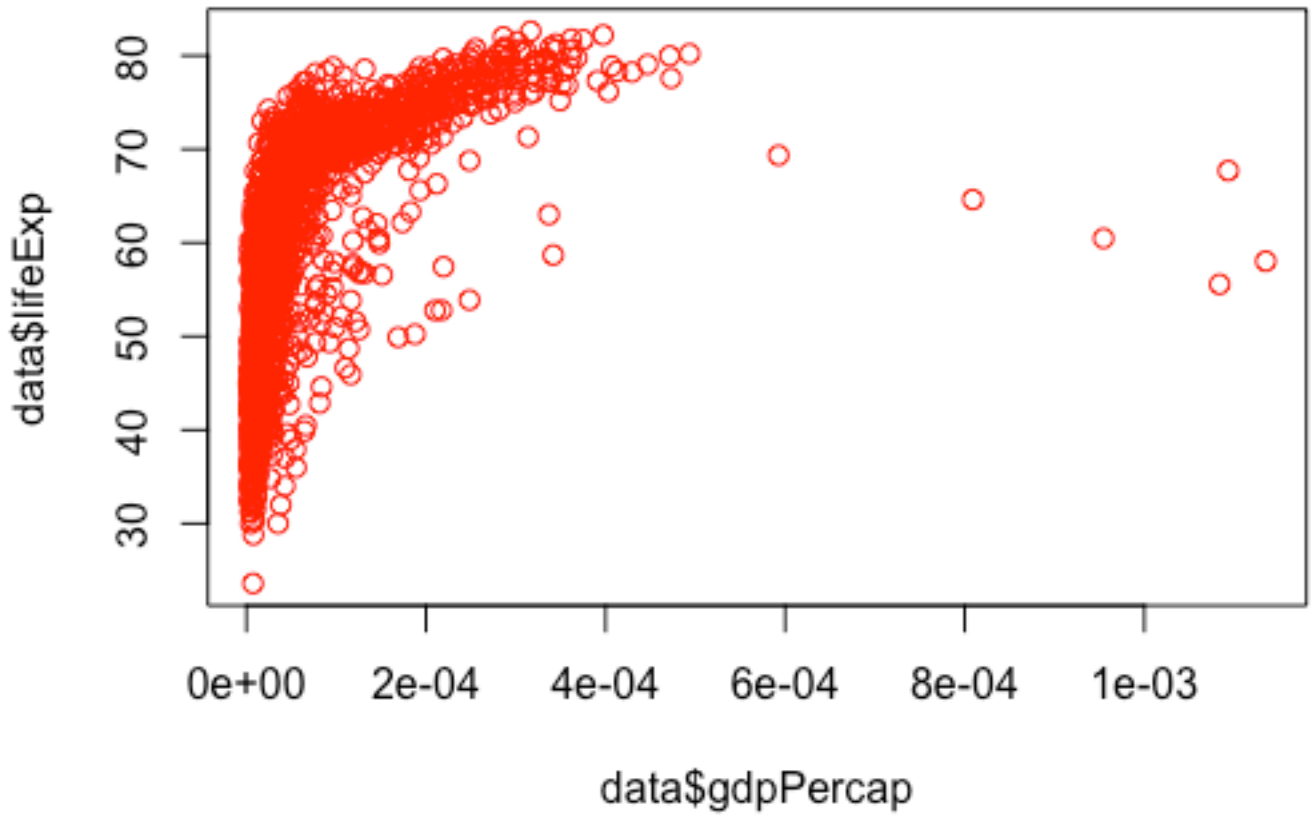
```
barplot(height, ...)

## Default S3 method:
barplot(height, width = 1, space = NULL,
        names.arg = NULL, legend.text = NULL, beside = FALSE,
        horiz = FALSE, density = NULL, angle = 45,
        col = NULL, border = par("fg"),
        main = NULL, sub = NULL, xlab = NULL, ylab = NULL,
        xlim = NULL, ylim = NULL, xpd = TRUE, log = "",
        axes = TRUE, axisnames = TRUE,
        cex.axis = par("cex.axis"), cex.names = par("cex.axis"),
        inside = TRUE, plot = TRUE, axis.lty = 0, offset = 0,
        add = FALSE, args.legend = NULL, ...)
```

Arguments

`height`	either a vector or matrix of values describing the bars which make up the plot. If `height` is a vector, the plot consists of a sequence of rectangular bars with heights given by the values in the vector. If `height` is a matrix and `beside` is `FALSE` then each bar of the plot corresponds to a column of `height`, with the values in the column giving the heights of stacked sub-bars making up the bar. If `height` is a matrix and `beside` is `TRUE`, then the values in each column are juxtaposed rather than stacked.
`width`	optional vector of bar widths. Re-cycled to length the number of bars drawn. Specifying a single value will have no visible effect unless `xlim` is specified.

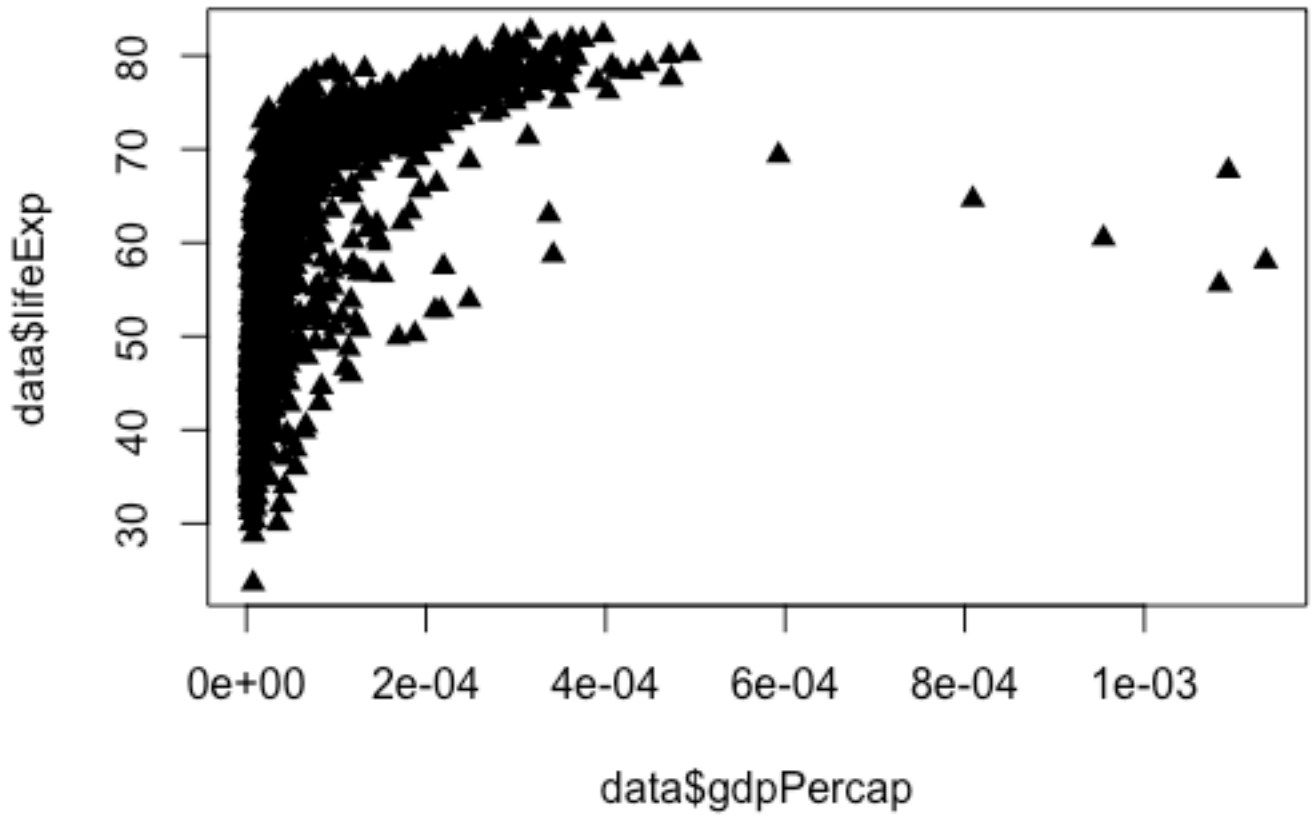
Some Arguments that Work for Most High-level Functions

```
plot(data$gdpPercap, data$lifeExp, col="red")
```



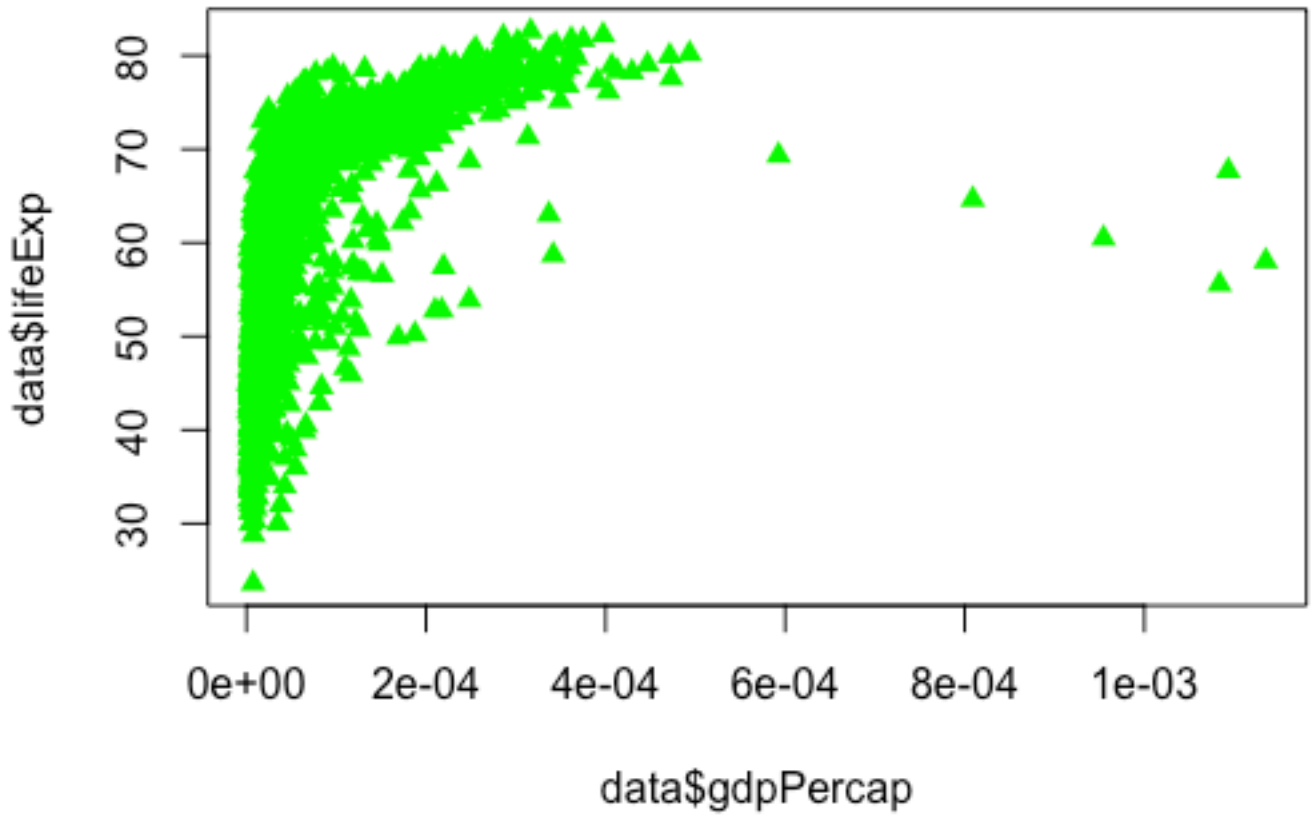
Some Arguments that Work for Most High-level Functions

```
plot(data$gdpPercap, data$lifeExp,  pch=17)
```



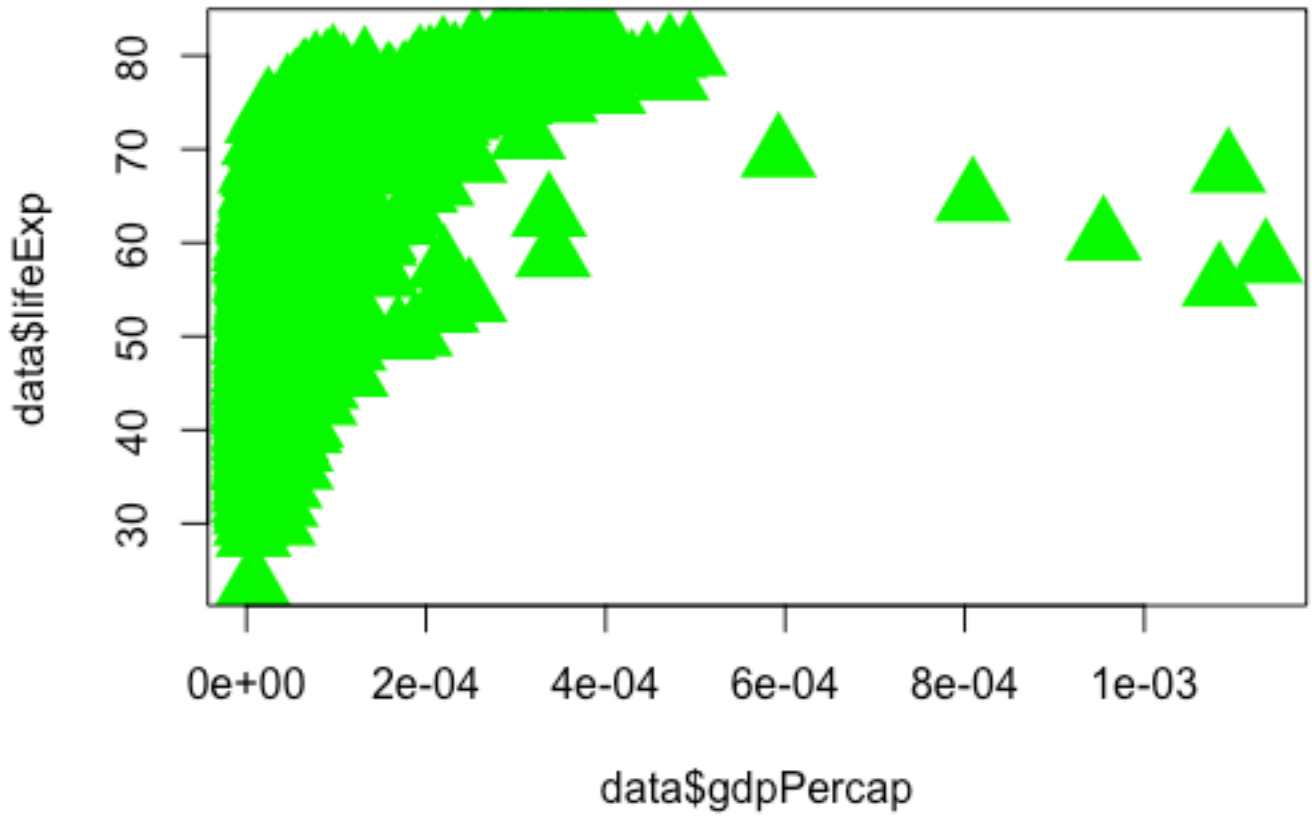
Some Arguments that Work for Most High-level Functions

```
plot(data$gdpPercap, data$lifeExp, pch=17, col="green")
```

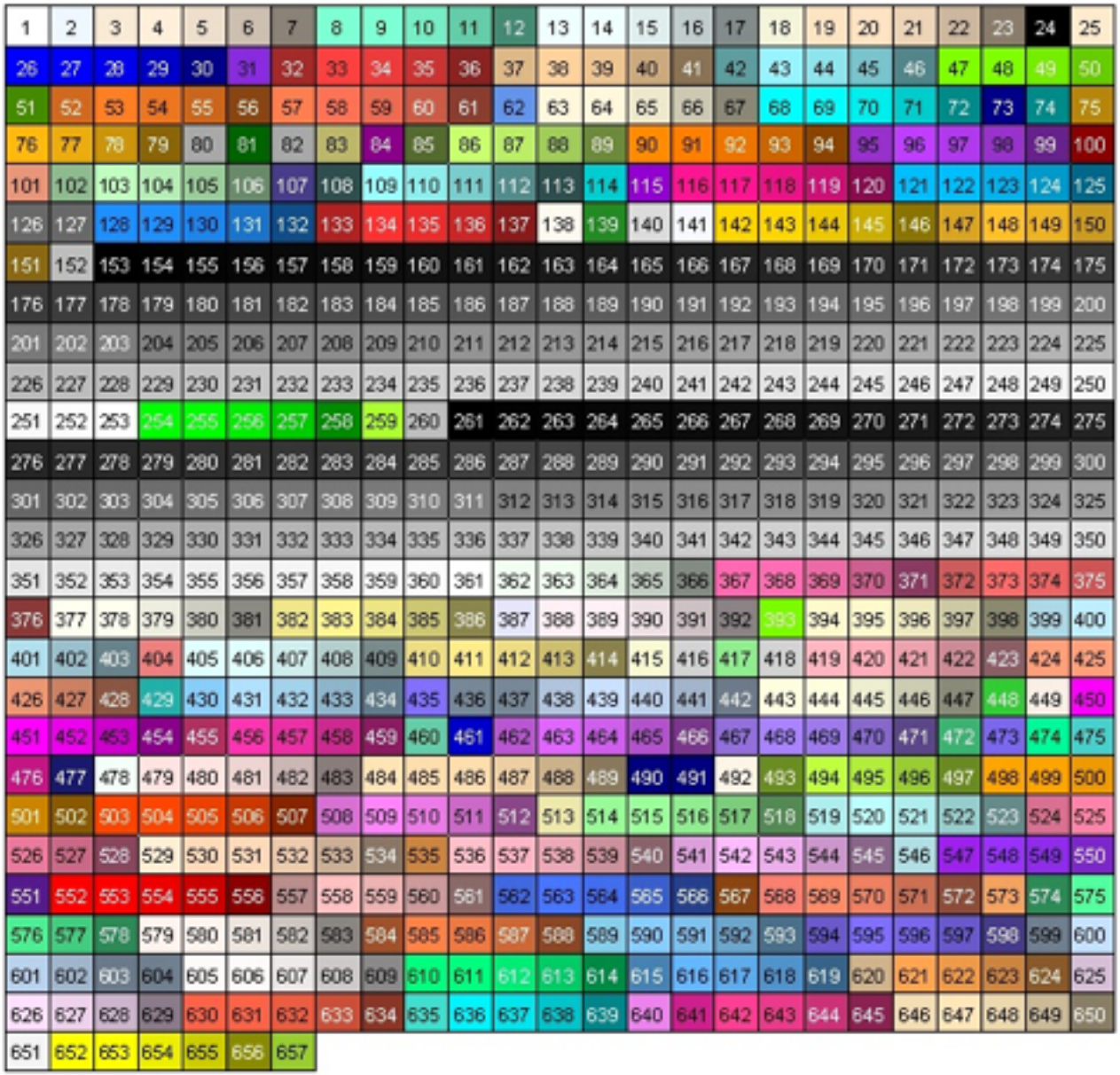


Some Arguments that Work for Most High-level Functions

```
plot(data$gdpPercap, data$lifeExp, pch=17, col="green", cex=3)
```



R colors

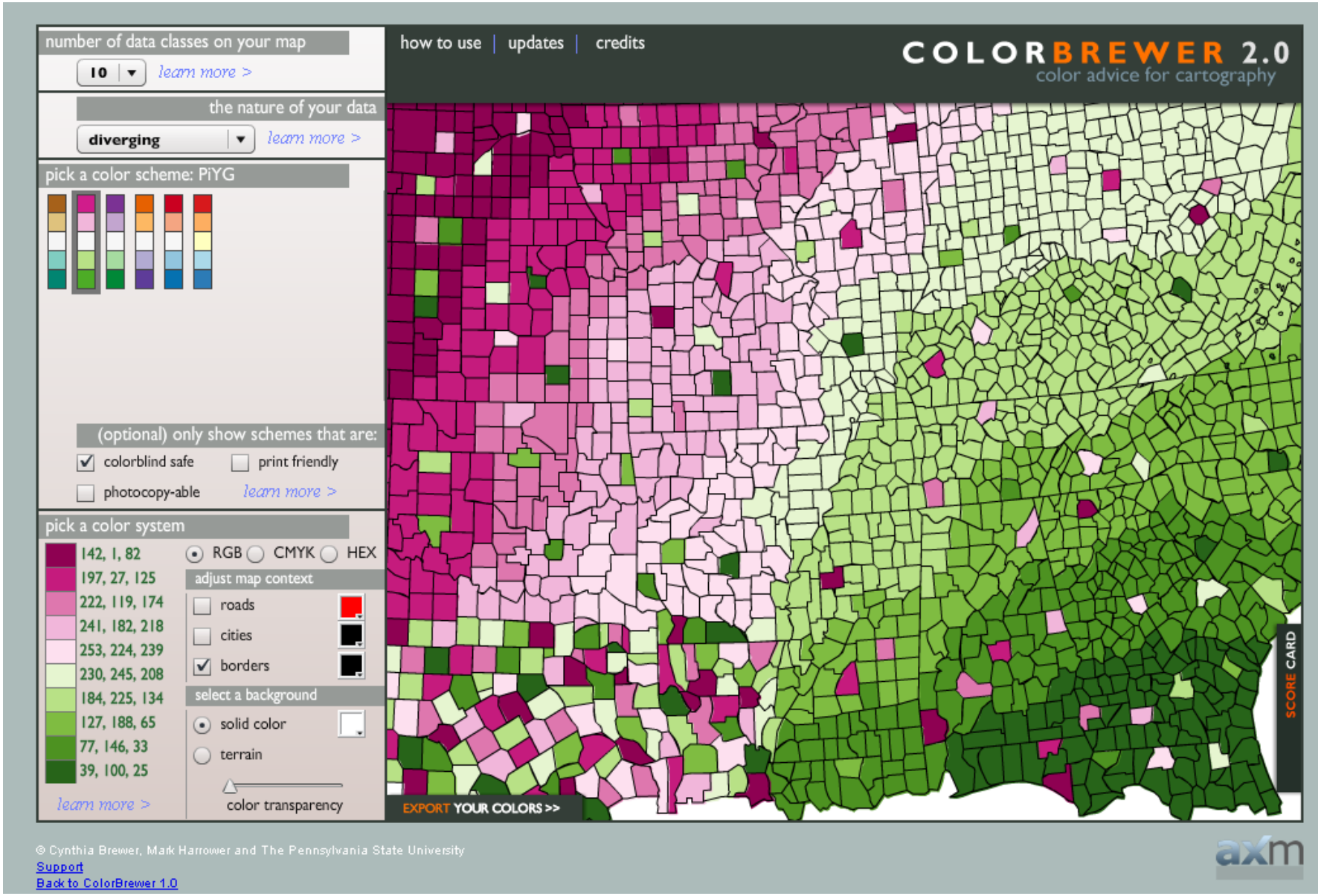

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75
76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150
151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175
176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200
201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225
226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250
251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275
276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300
301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325
326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350
351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375
376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400
401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425
426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475
476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500
501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525
526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550
551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575
576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600
601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650
651 652 653 654 655 656 657

How to find your favorite color

```
colors()[c(1, 280, 637)]
```

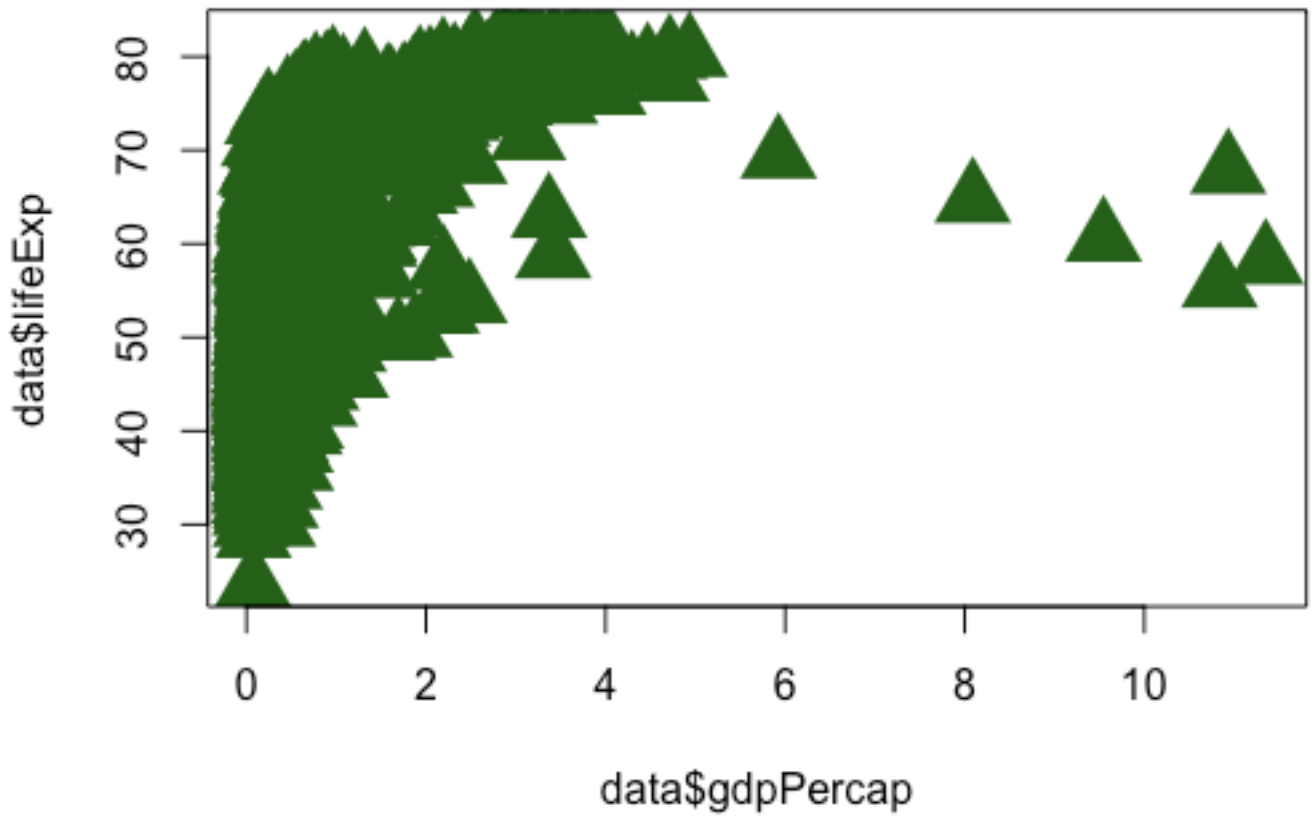
```
[1] "white" "grey19" "turquoise2"
```

colorbrewer2.org

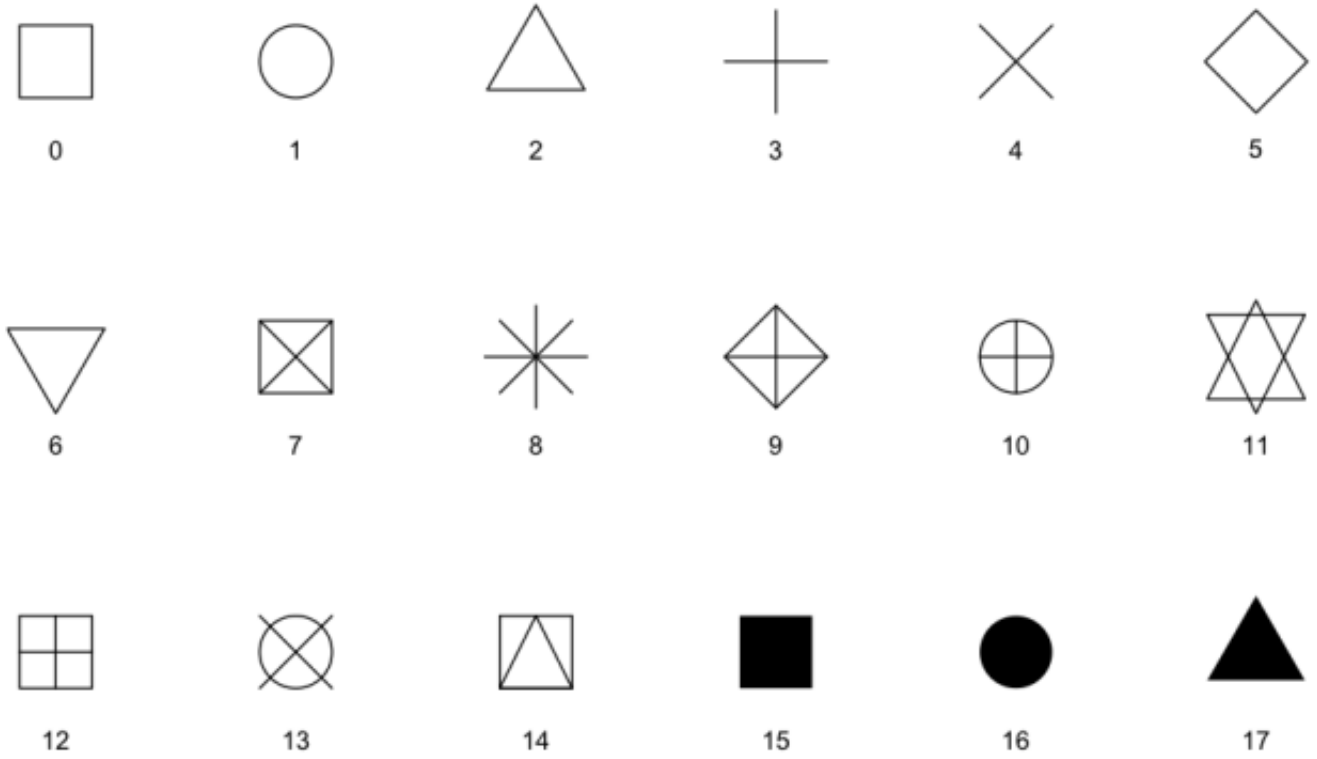


How to find your favorite color

```
plot(data$gdpPercap, data$lifeExp, pch=17, col=rgb(39, 100, 25,
max=255), cex=3)
```

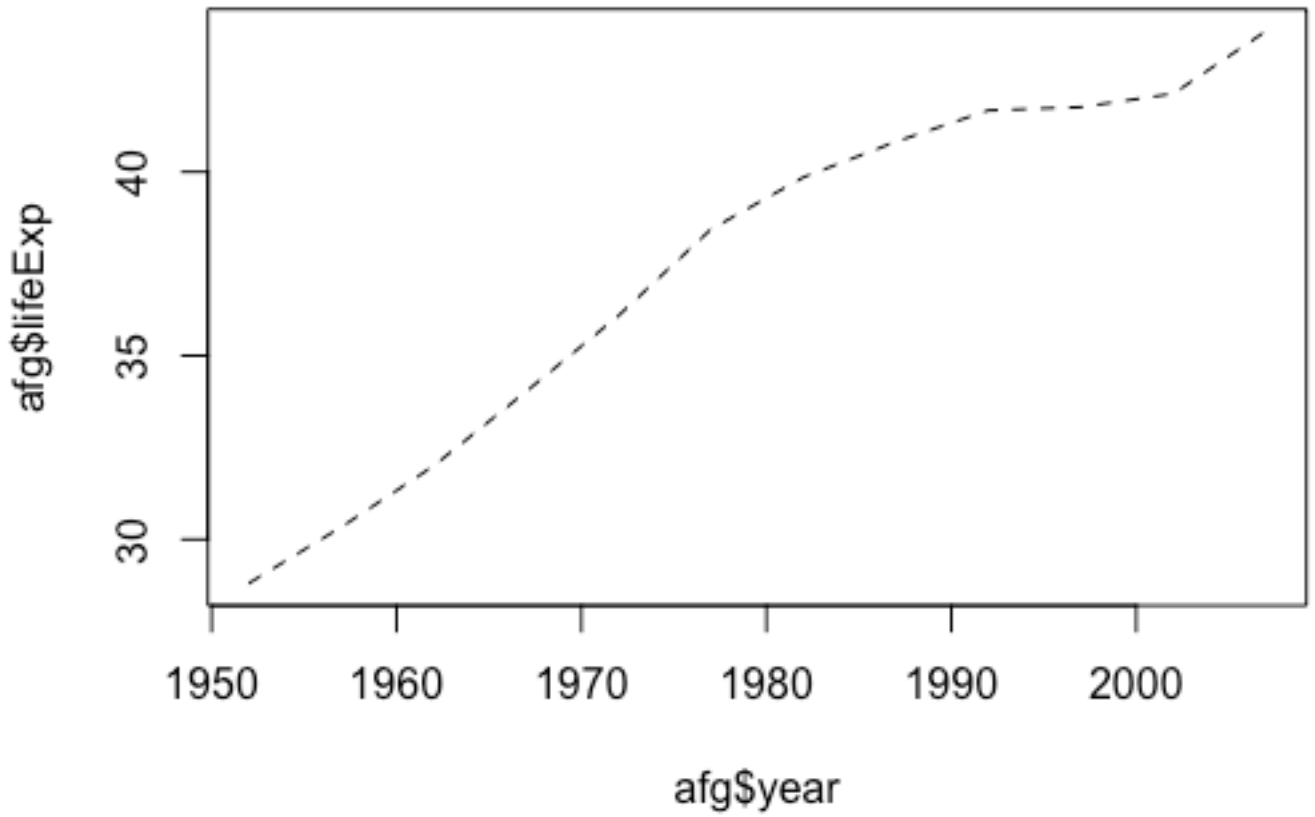


Plotting Characters



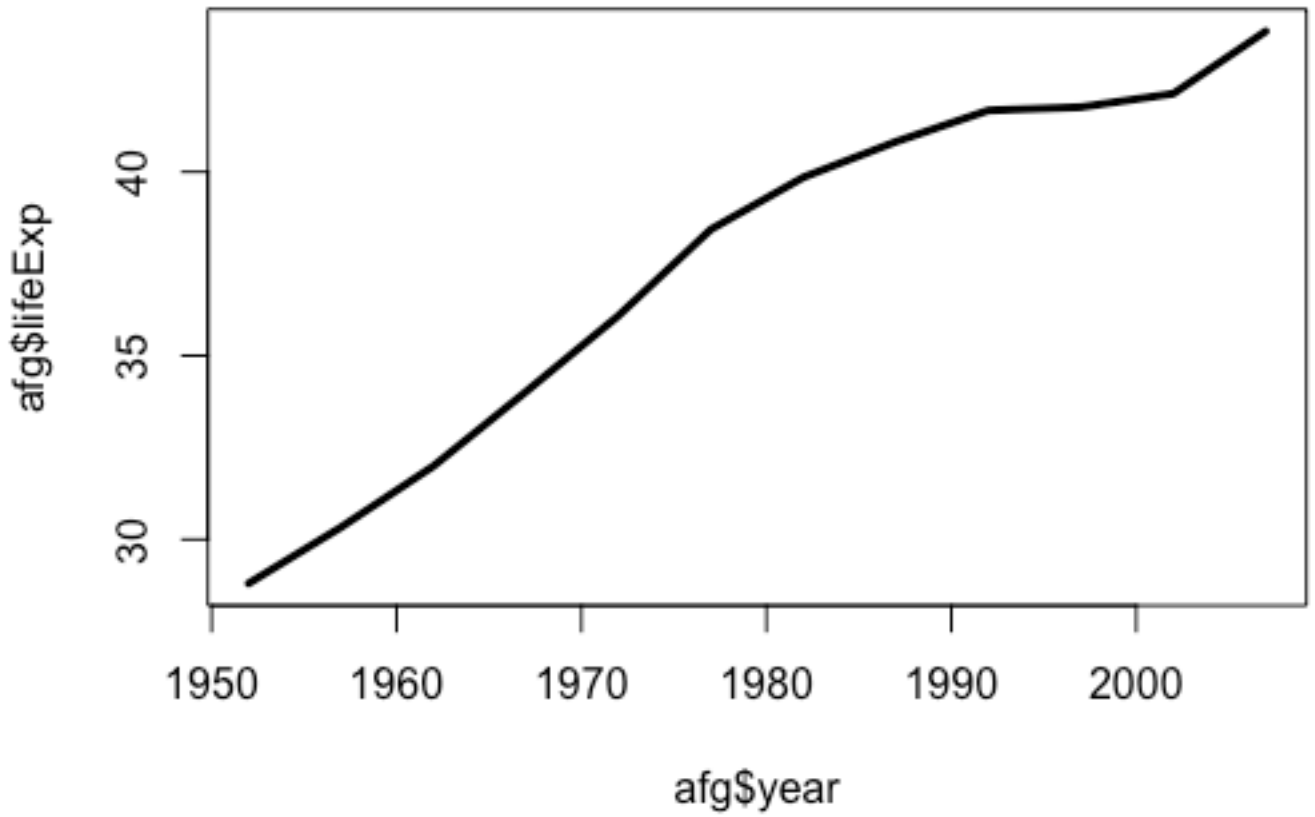
Some Arguments that Work for Most High-level Functions

```
plot(afg$year, afg$lifeExp, type="l", lty="dashed")
```



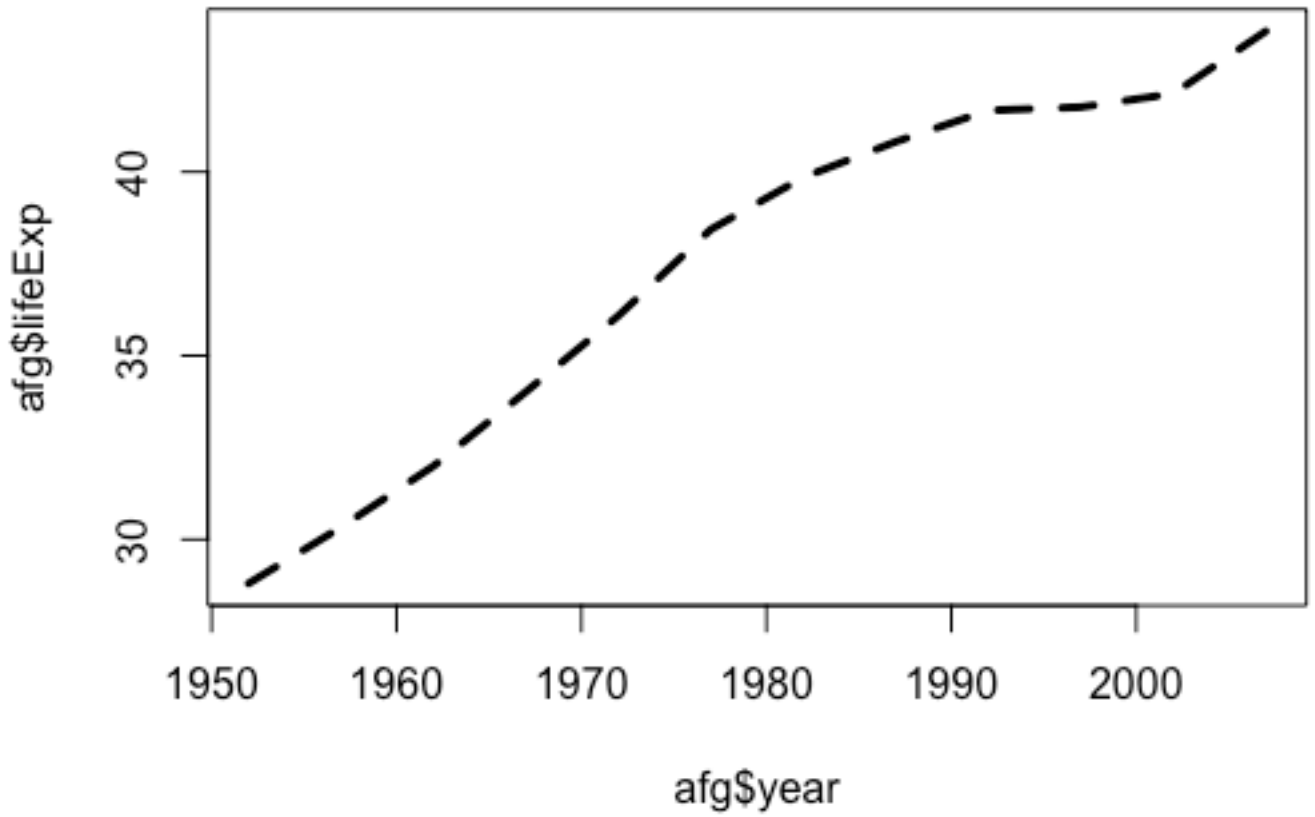
Some Arguments that Work for Most High-level Functions

```
plot(afg$year, afg$lifeExp, type="l", lwd=3)
```



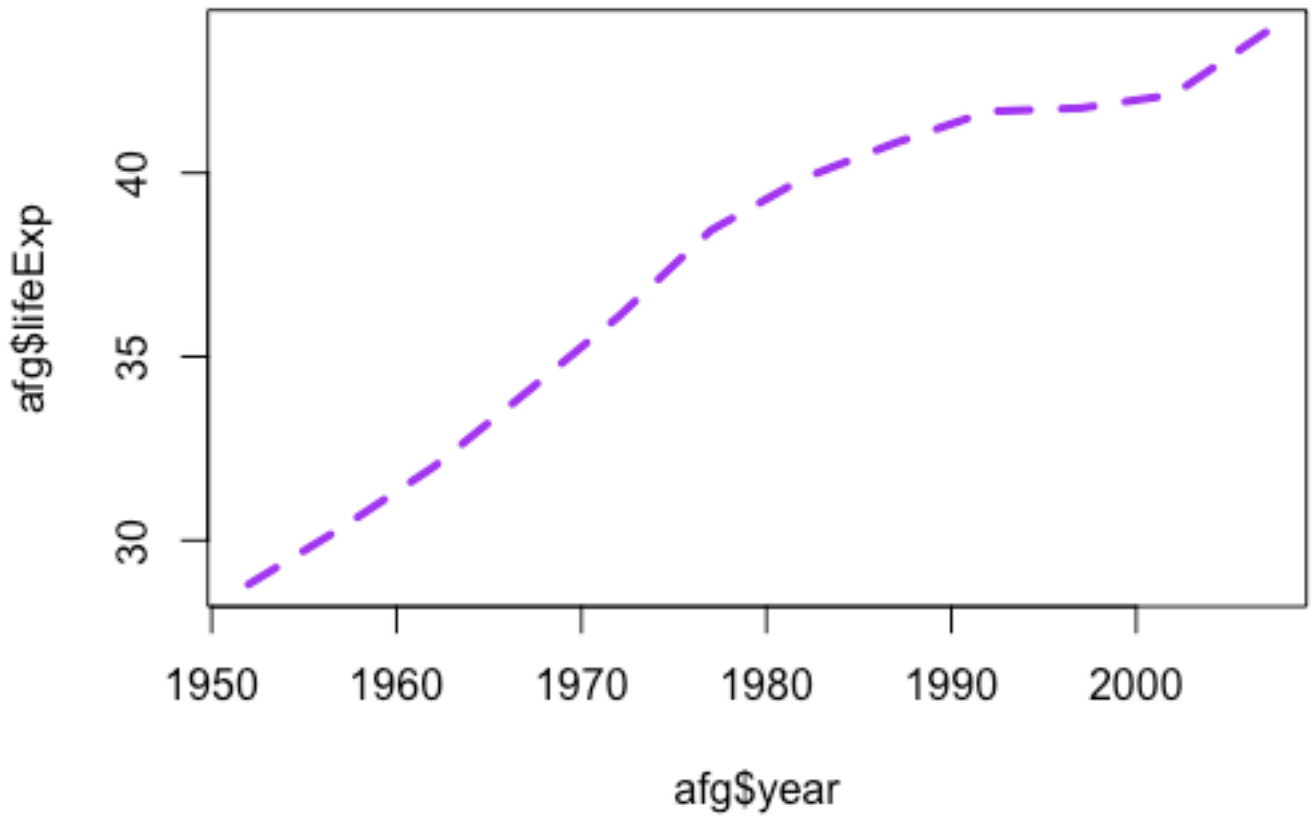
Some Arguments that Work for Most High-level Functions

```
plot(afg$year, afg$lifeExp, type="l", lwd=3, lty="dashed")
```



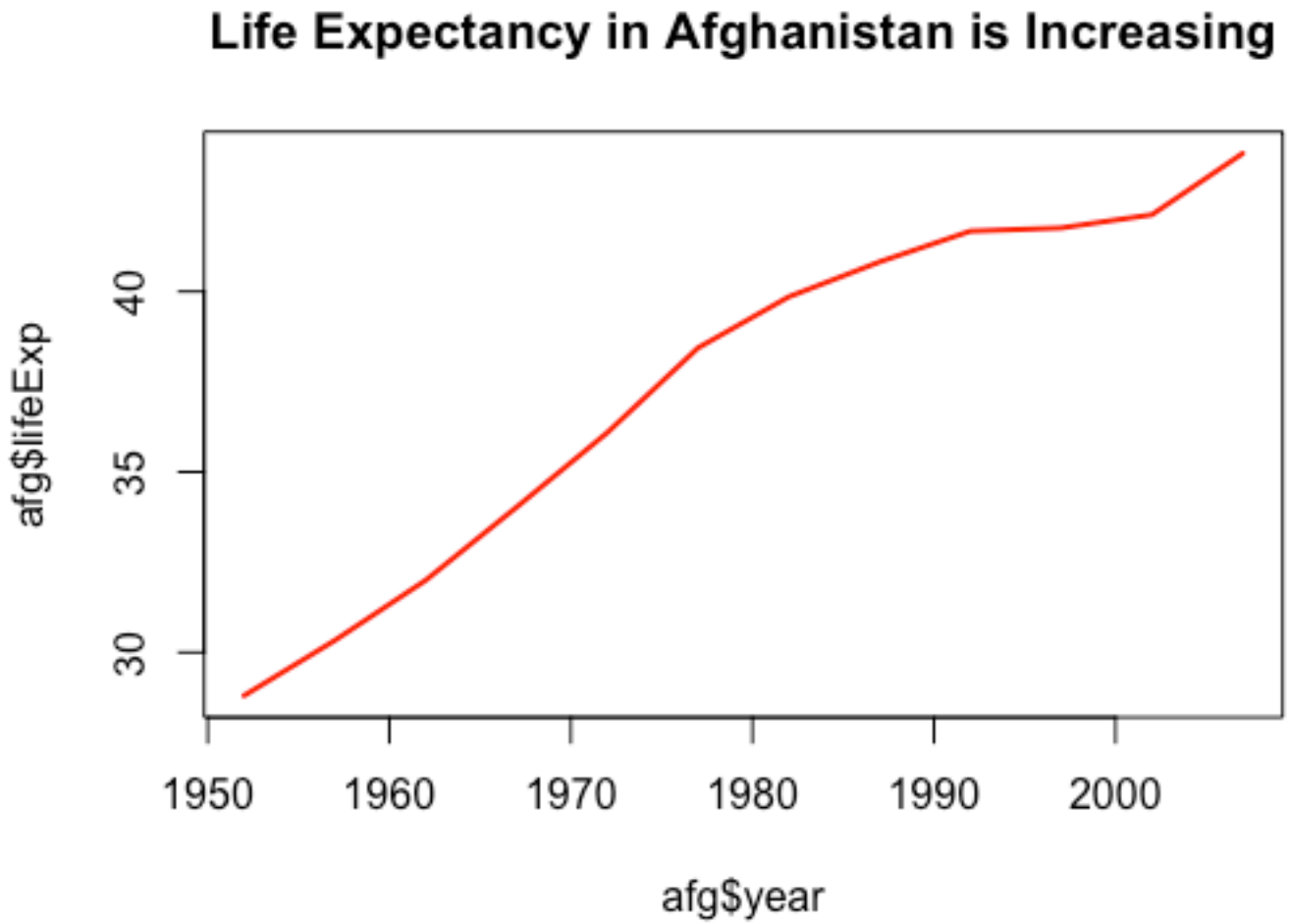
Some Arguments that Work for Most High-level Functions

```
plot(afg$year, afg$lifeExp, type="l", lwd=3, lty="dashed",
col="purple")
```



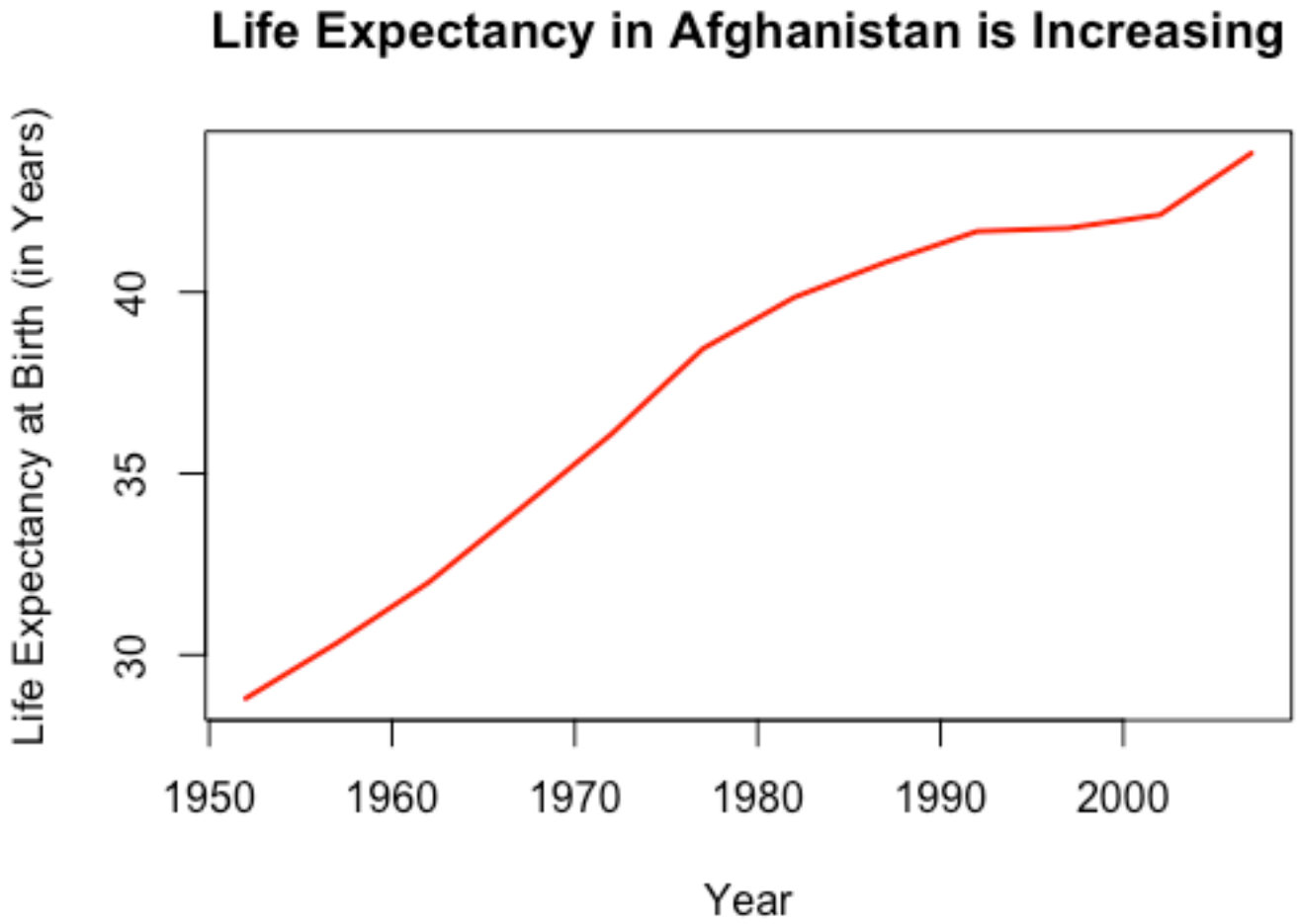
More Arguments: Title, Labels, Axes

```
plot(afg$year, afg$lifeExp, type="l", lwd=2, col="red",
main="Life Expectancy in Afghanistan is Increasing")
```



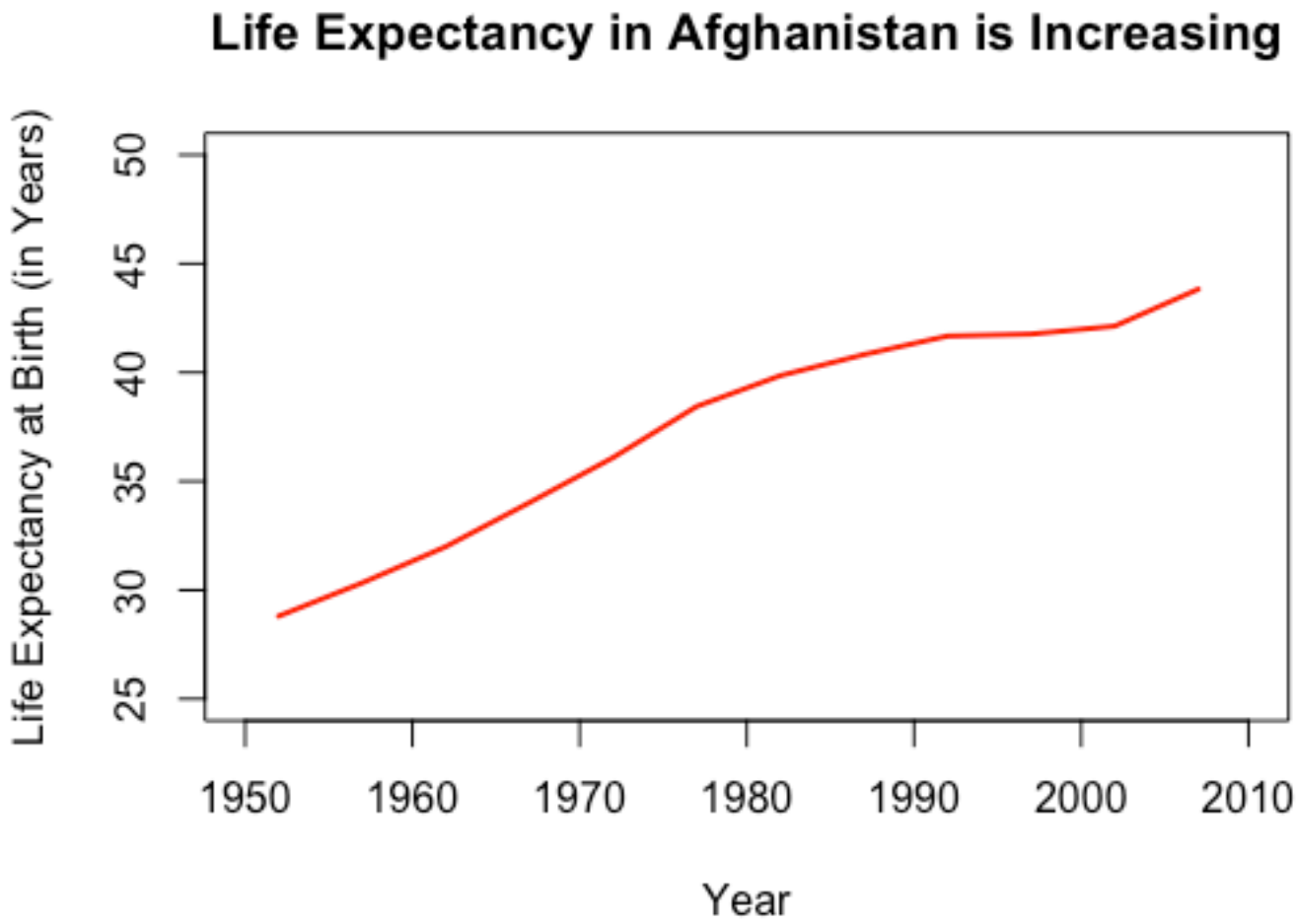
More Arguments: Title, Labels, Axes

```
plot(afg$year, afg$lifeExp, type="l", lwd=2, col="red",
main="Life Expectancy in Afghanistan is Increasing",
xlab="Year", ylab="Life Expectancy at Birth (in Years)")
```



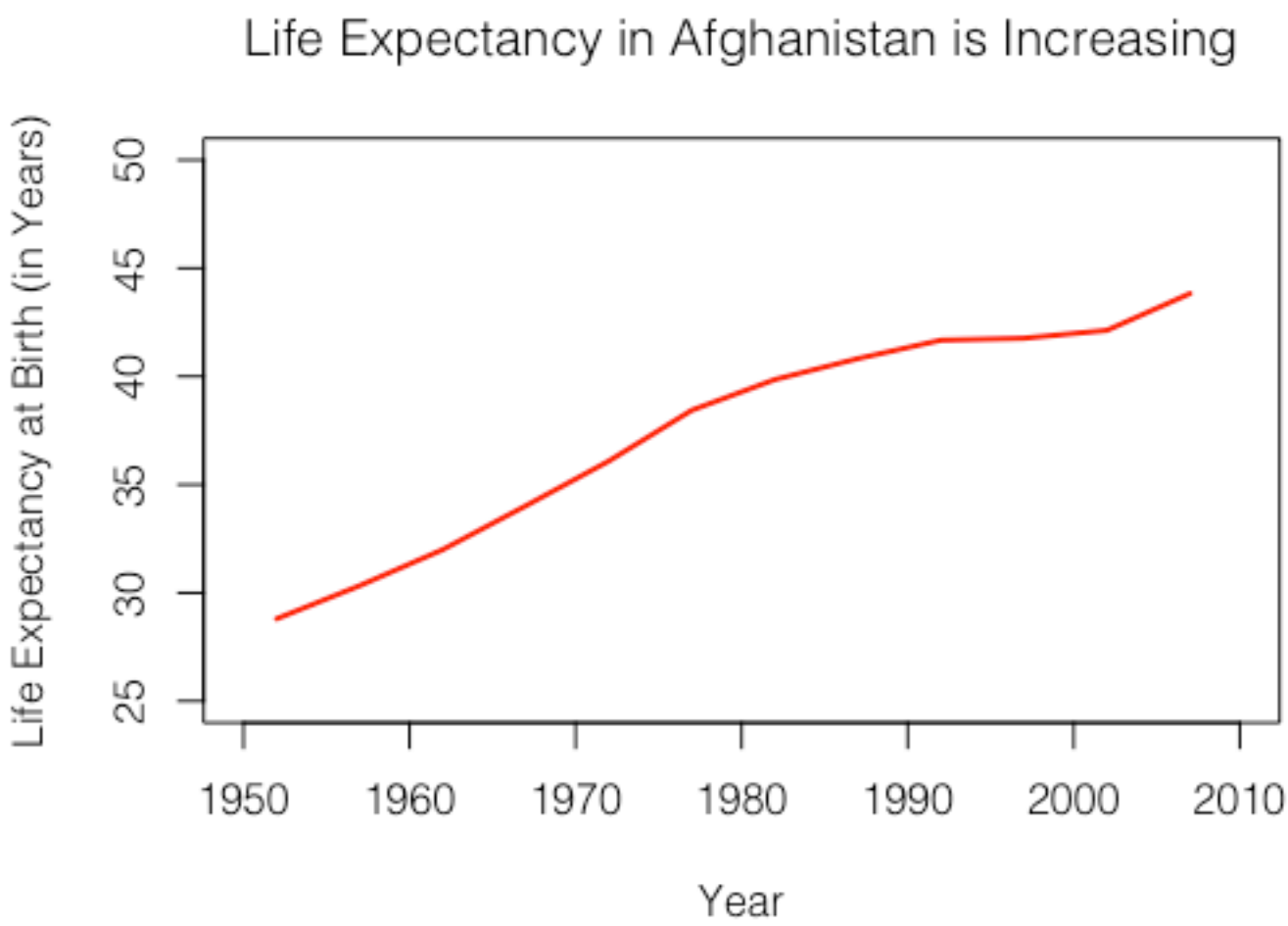
More Arguments: Title, Labels, Axes

```
plot(afg$year, afg$lifeExp, type="l", lwd=2, col="red",… ,
ylim=c(25, 50), xlim=c(1950, 2010))
```



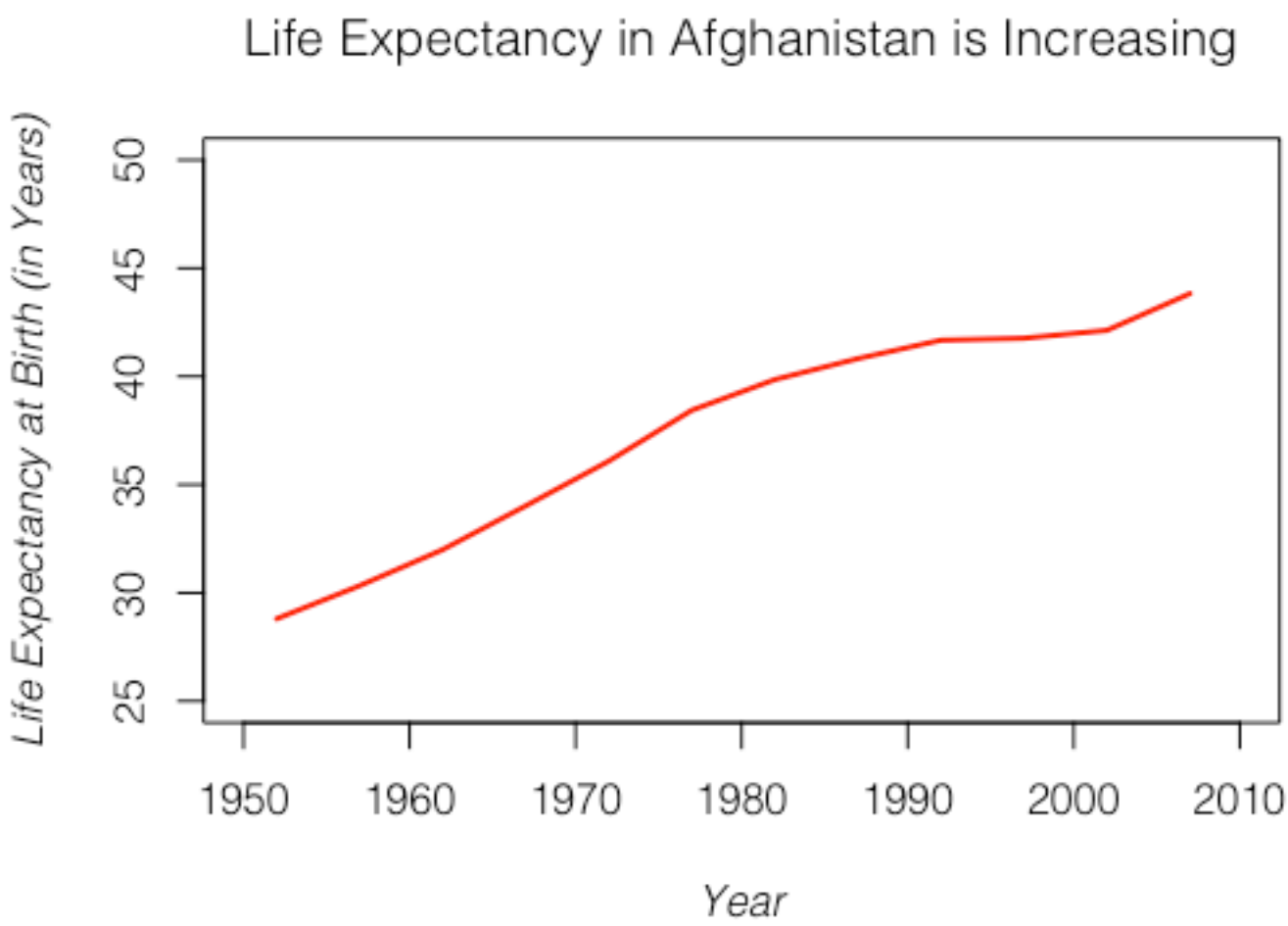
More Arguments: Title, Labels, Axes

```
plot(afg$year, afg$lifeExp, type="l", lwd=2, col="red", … ,
family="Helvetica Light")
```



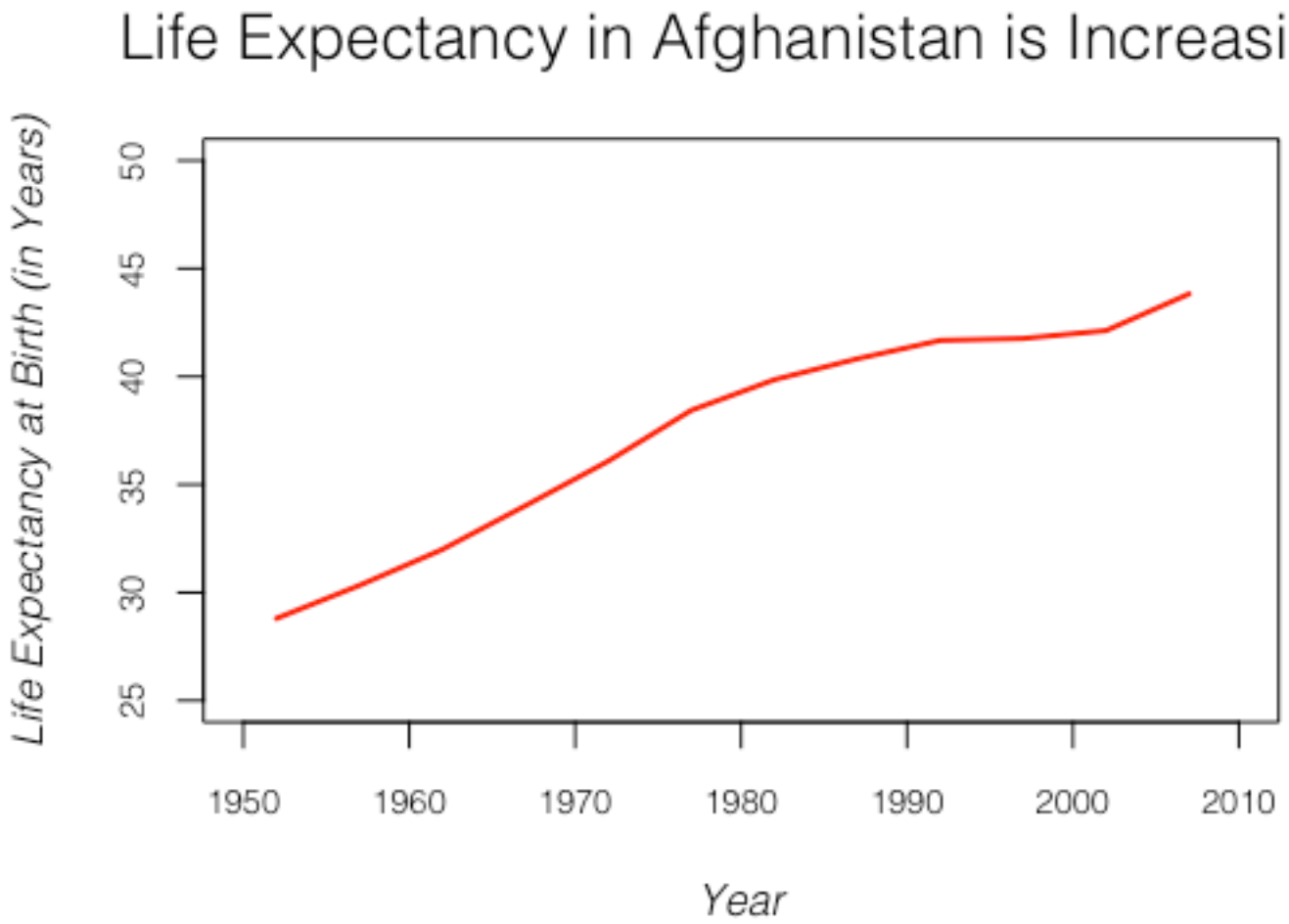
More Arguments: Title, Labels, Axes

```
plot(afg$year, afg$lifeExp, type="l", lwd=2, col="red", … ,
family="Helvetica Light", font.main=1, font.lab=3)
```

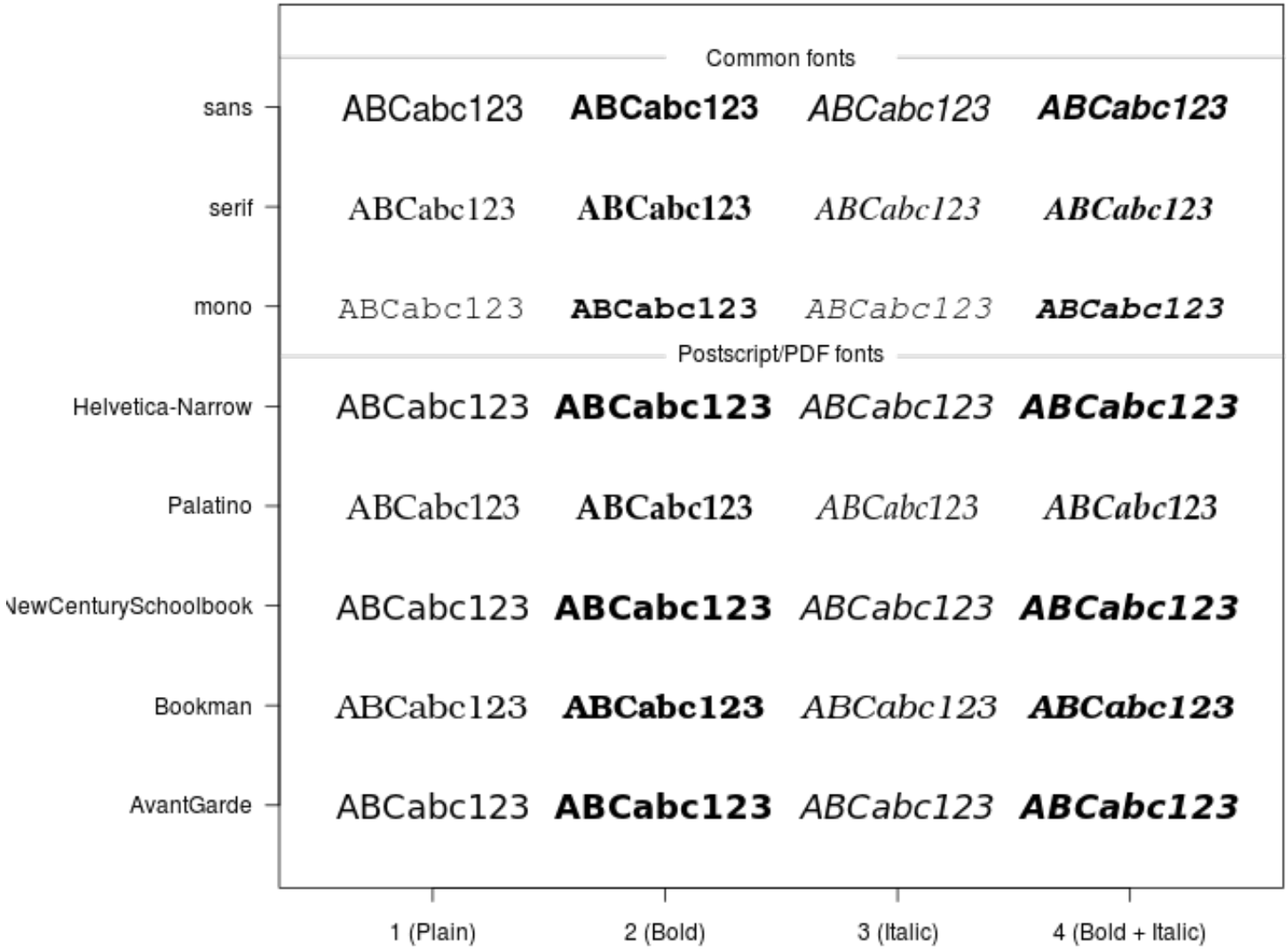


More Arguments: Title, Labels, Axes

```
plot(afg$year, afg$lifeExp, type="l", lwd=2, col="red", … ,
family="Helvetica Light", font.main=1, font.lab=3, cex.main=1.5,
cex.lab=1, cex.axis=.8)
```



ont Types and Families



The `par()` Function

Many arguments can be applied directly to high level functions such as `plot()`

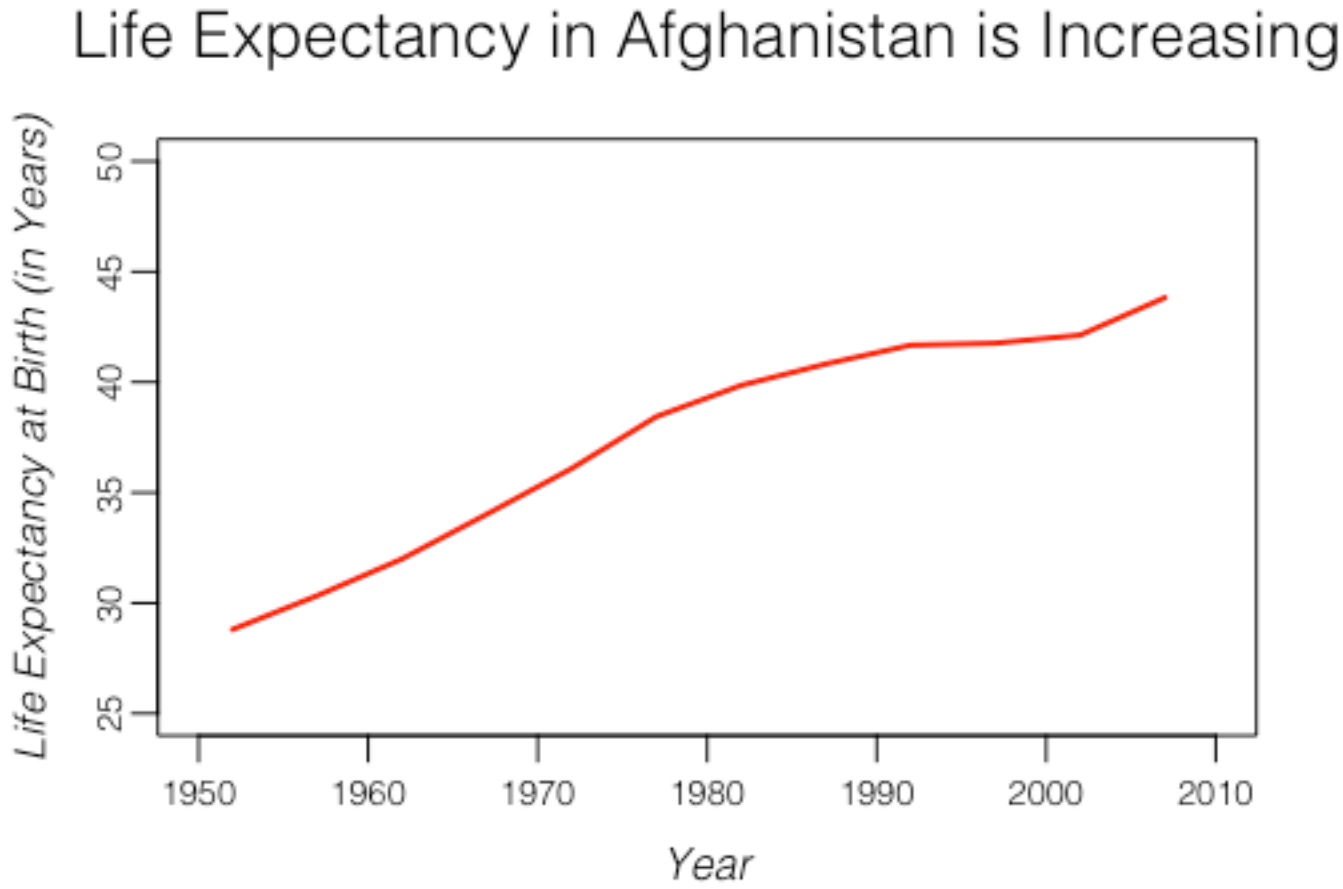
See help for all possible arguments and how to define them: `?plot()`

Arguments can also be specified using the `par()` function

This sets parameters permanently for a session and applies them to all plots

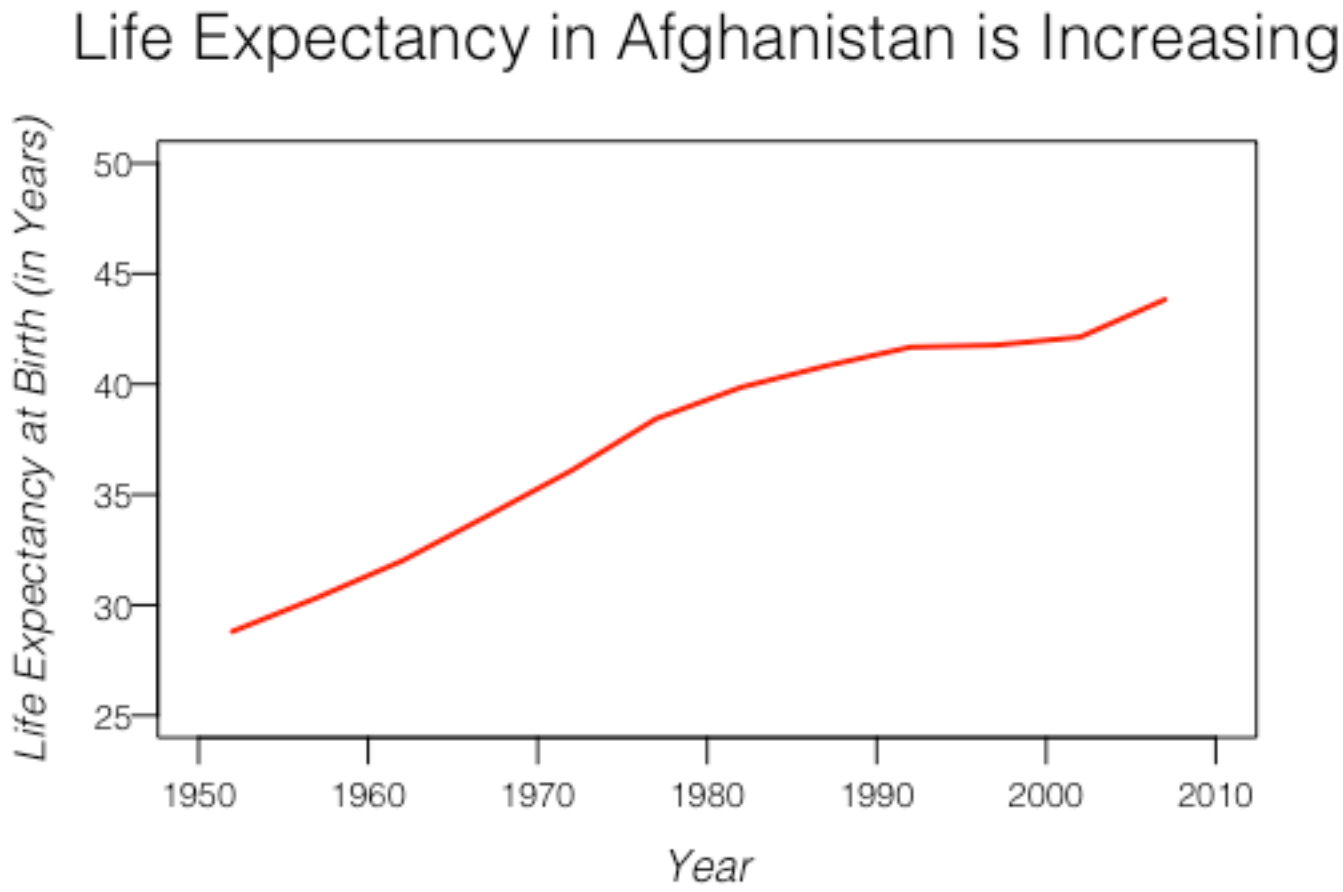
Some more arguments with `par()`

```
par(mgp=c(2, .5, 0))
```



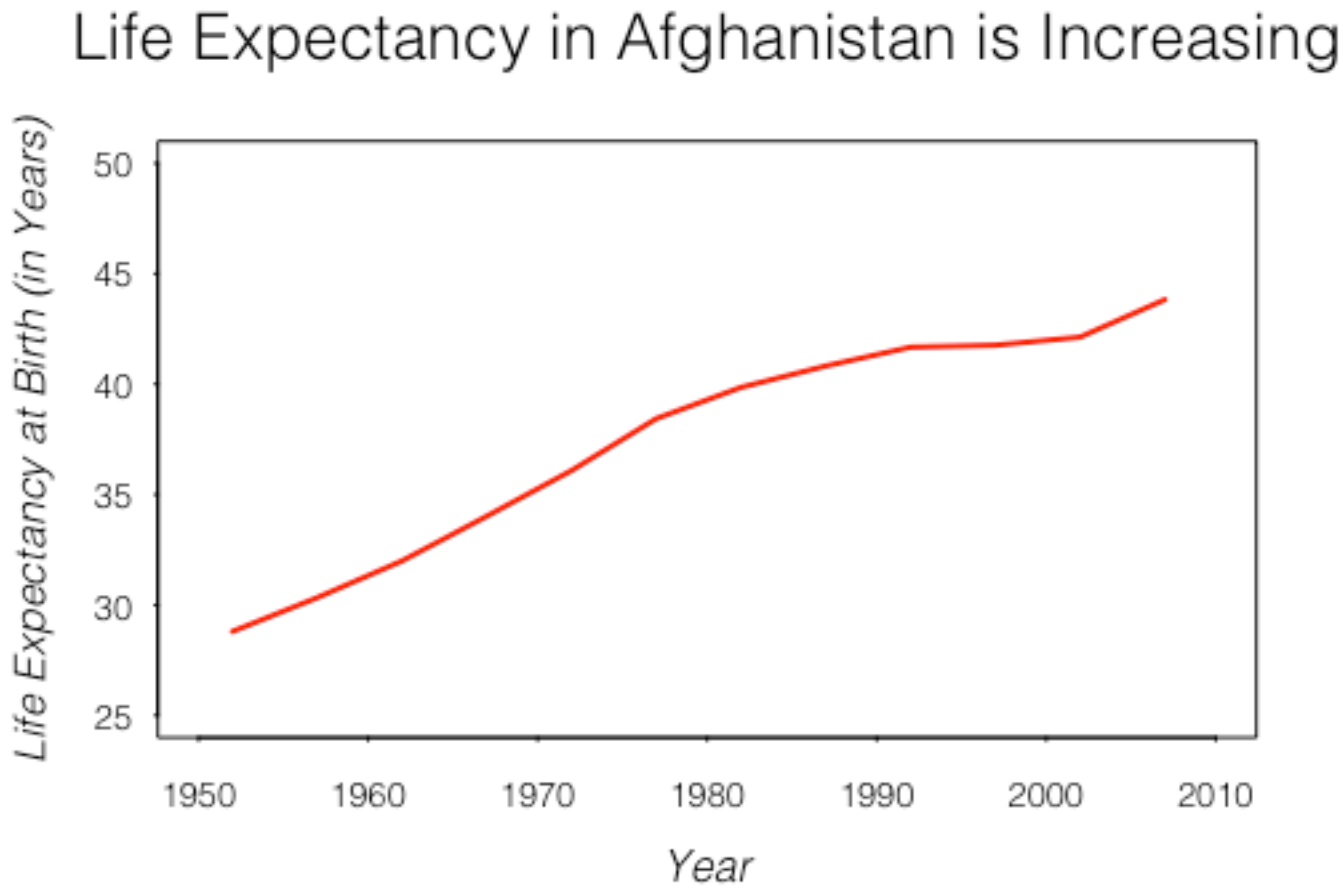
Some more arguments with `par()`

```
par(mgp=c(2, .5, 0), las=1)
```



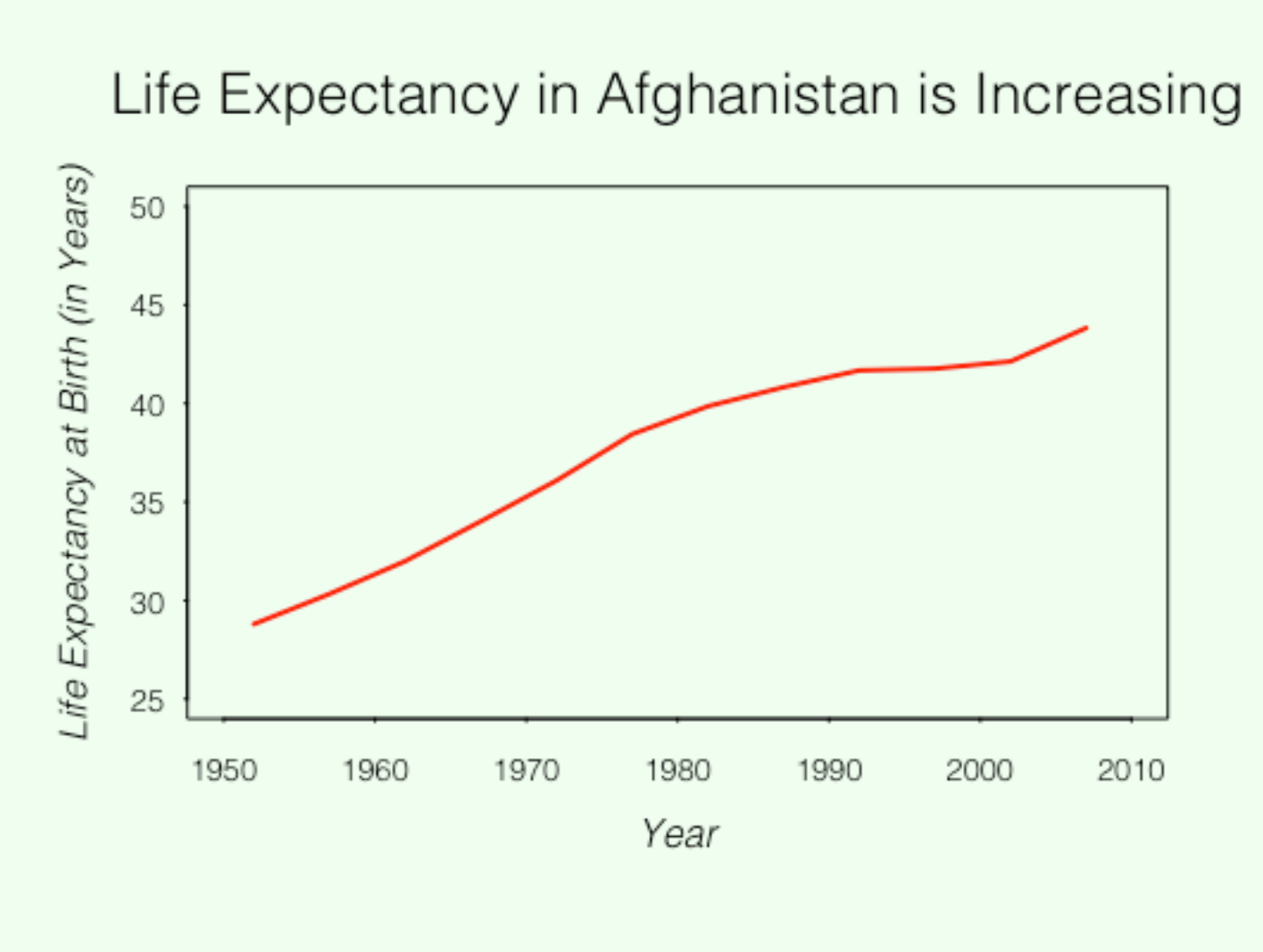
Some more arguments with `par()`

```
par(mgp=c(2, .5, 0), las=1, tck=-0.005)
```



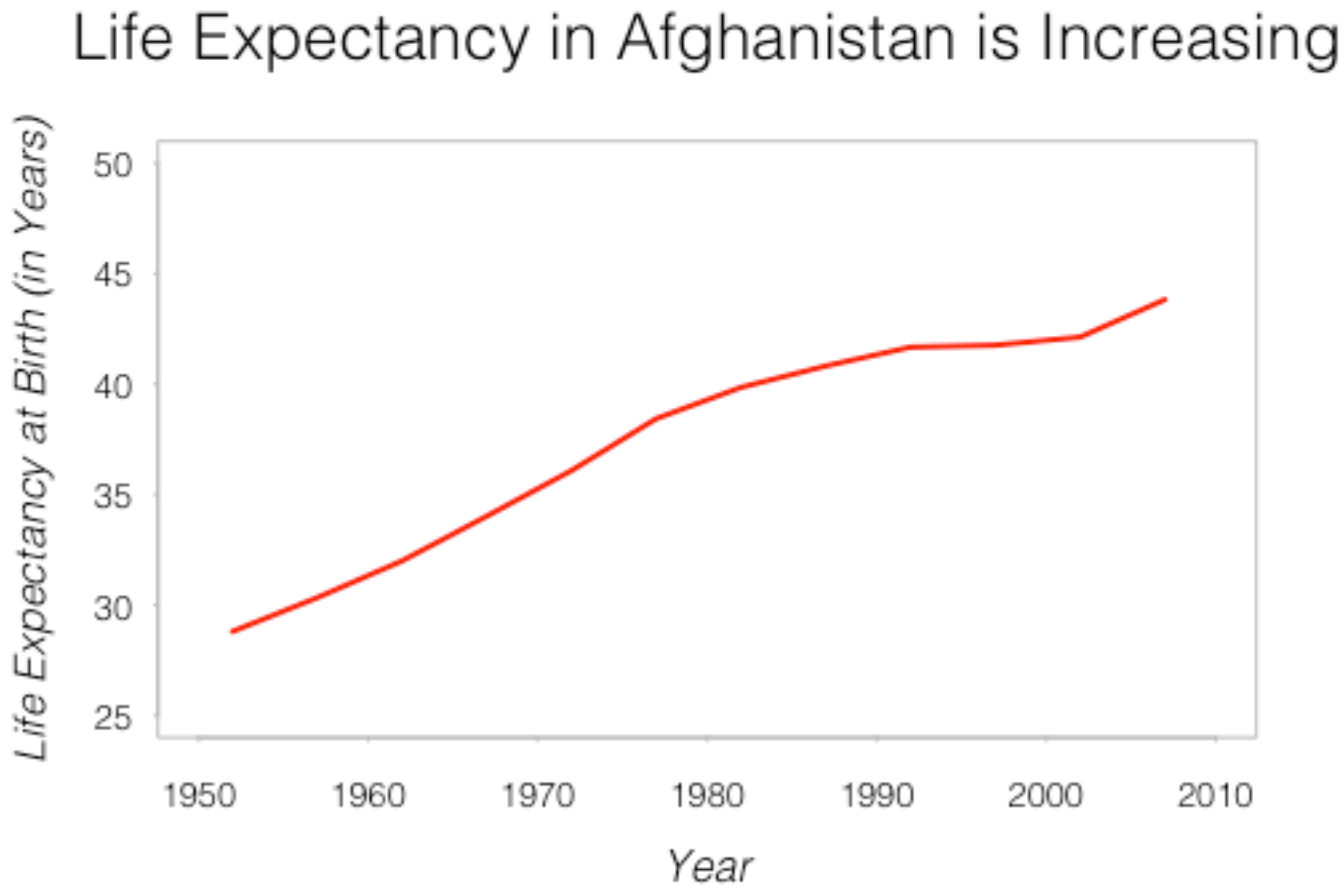
Some more arguments with `par()`

```
par(mgp=c(2, .5, 0), las=1, tck=-0.005, bg="honeydew")
```



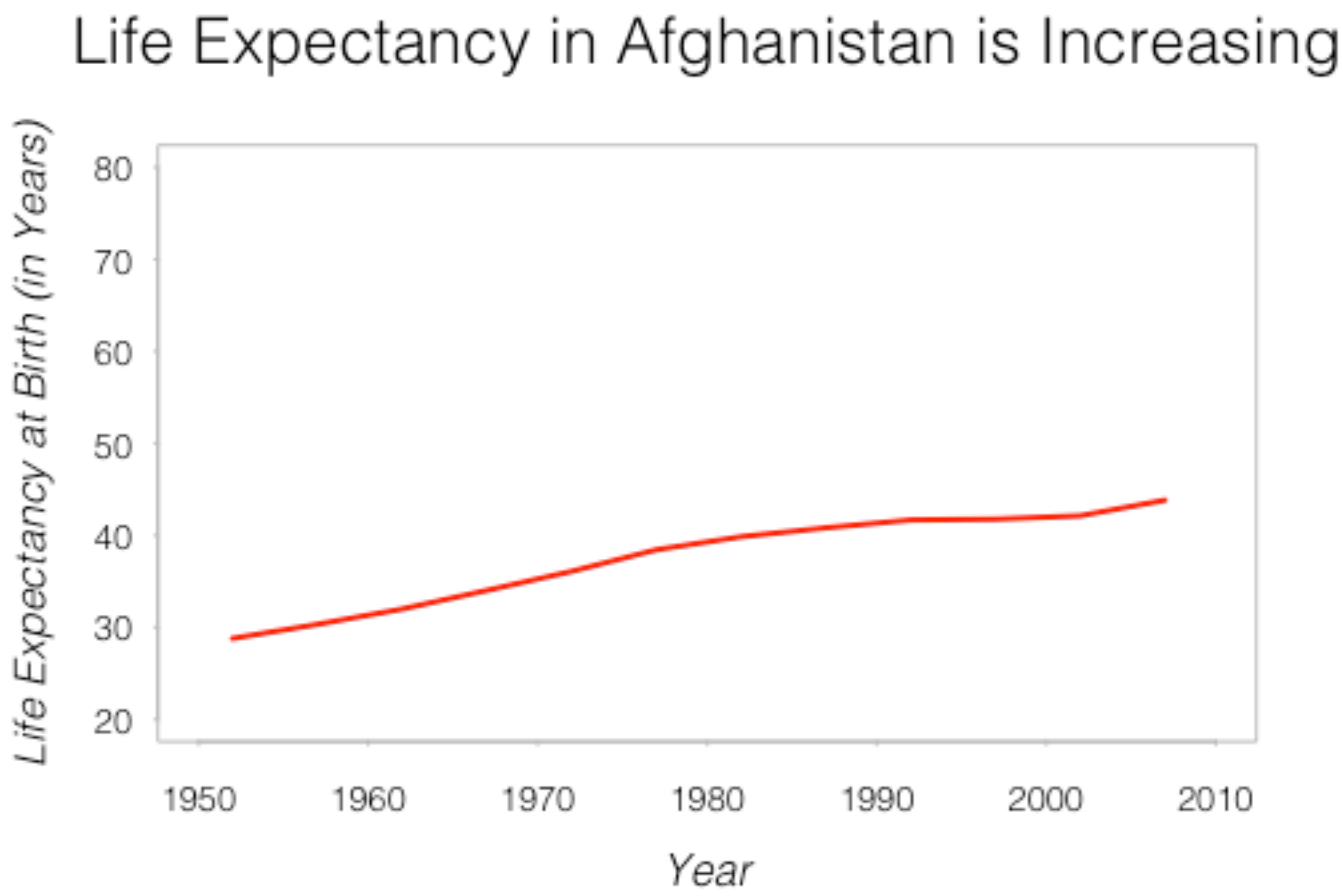
Some more arguments with `par()`

```
par(mgp=c(2, .5, 0), las=1, tck=-0.005, fg="grey")
```



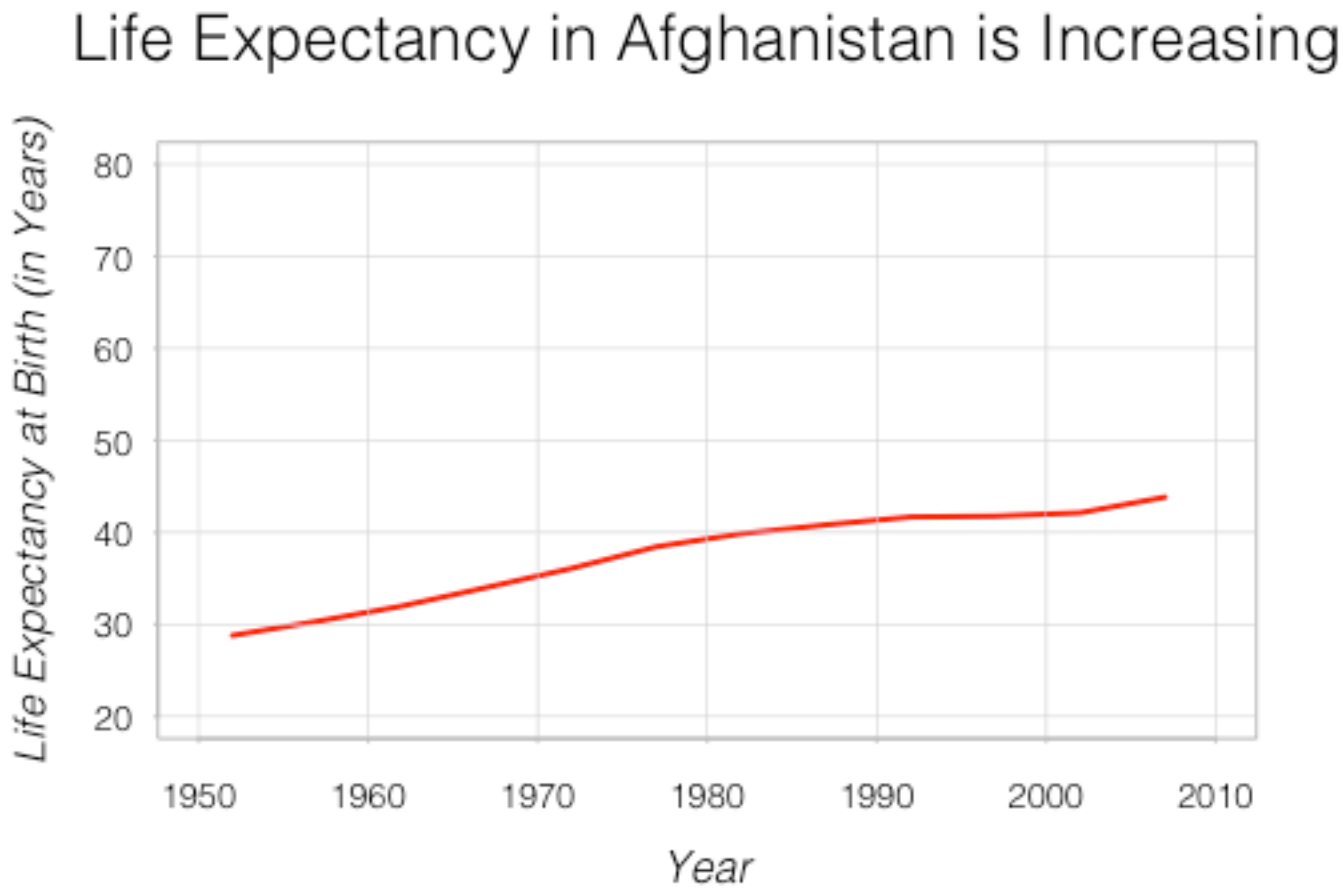
Adding Elements to an Existing Plot

Some Low-level Functions



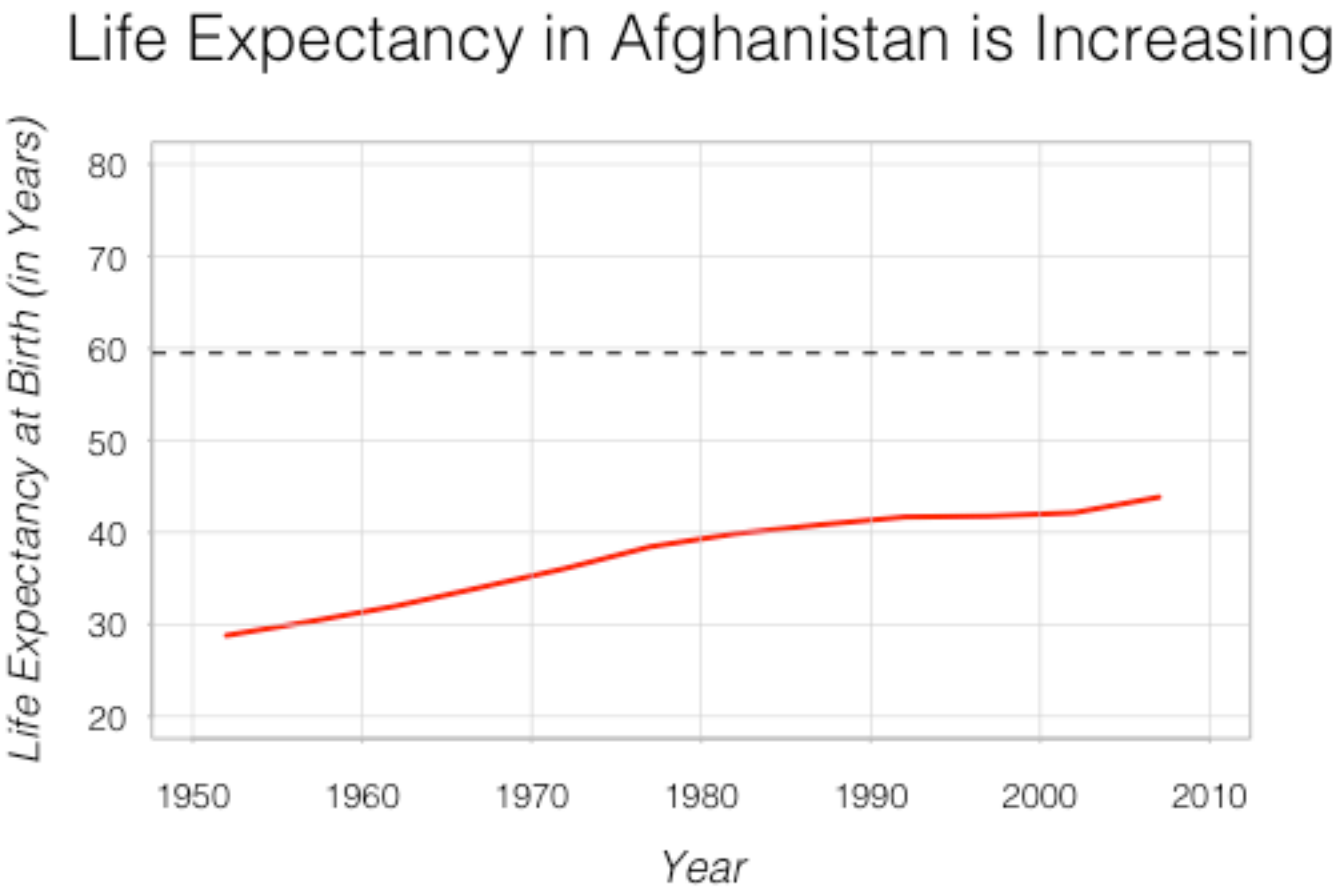
Some Low-level Functions

```
grid(lty=1, lwd=.5)
```



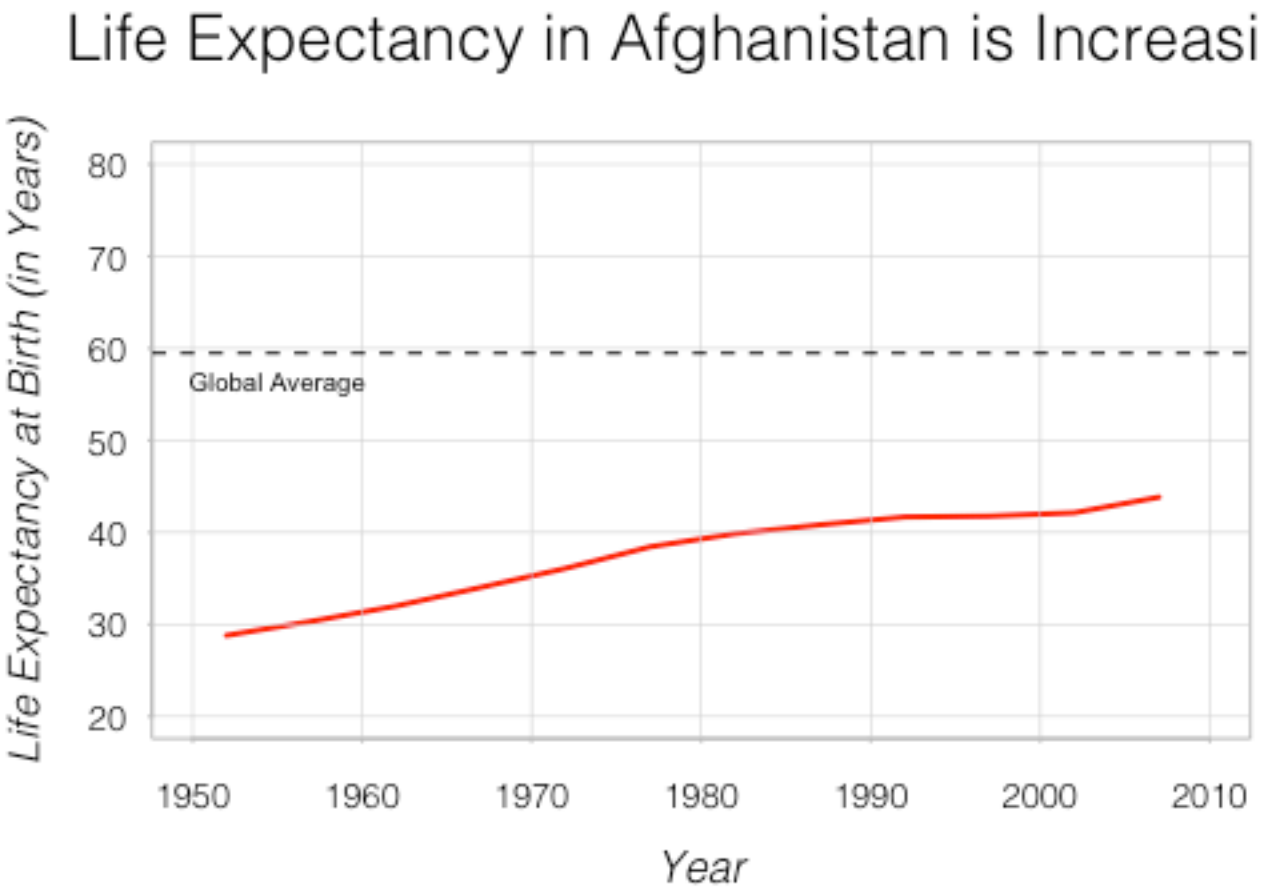
Some Low-level Functions

```
average.life <- mean(data$lifeExp)
abline(h=average.life, lty=2, col="black")
```



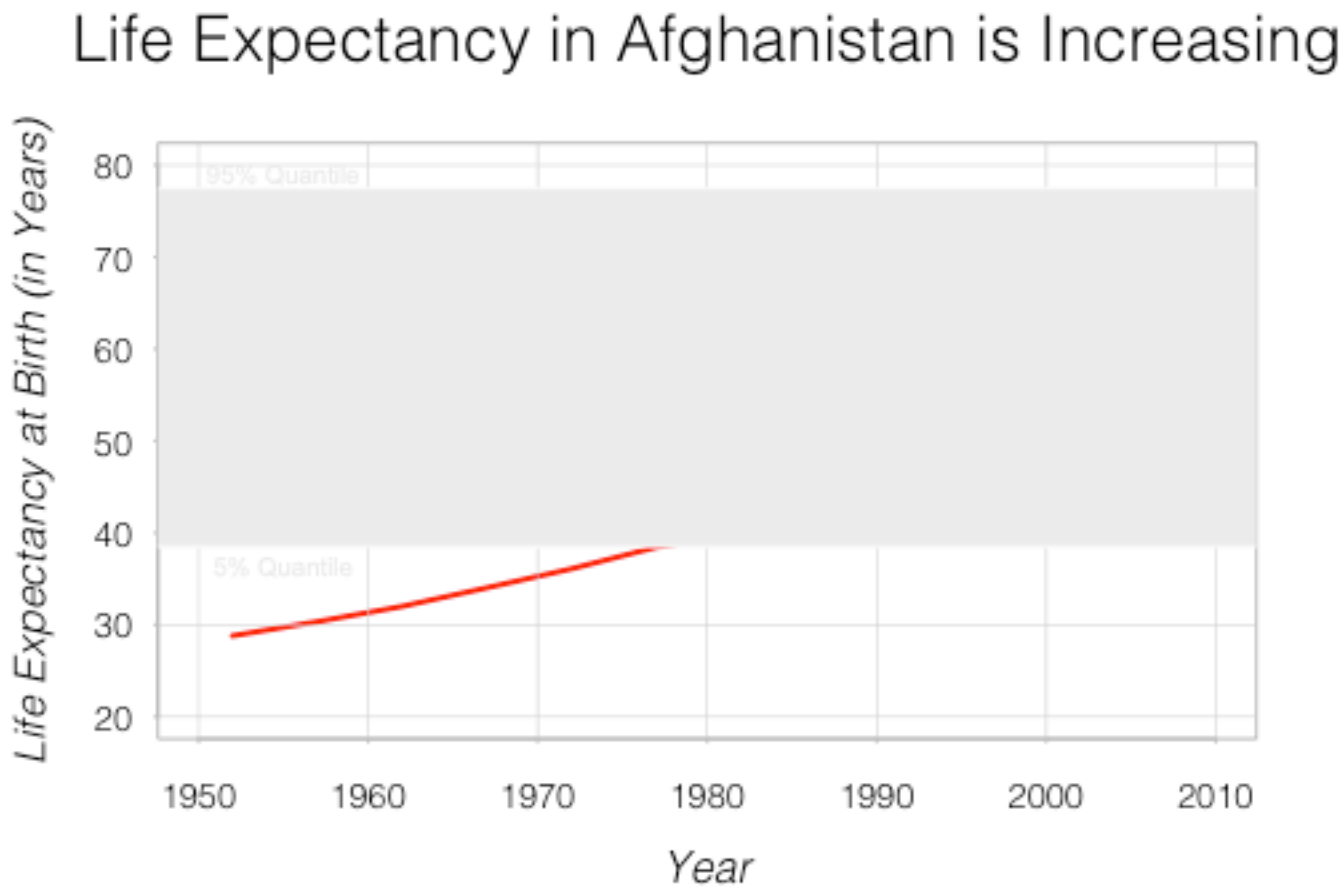
Some Low-level Functions

```
text(1955, 60, "Global Average", col="black", pos=1, cex=.6)
```



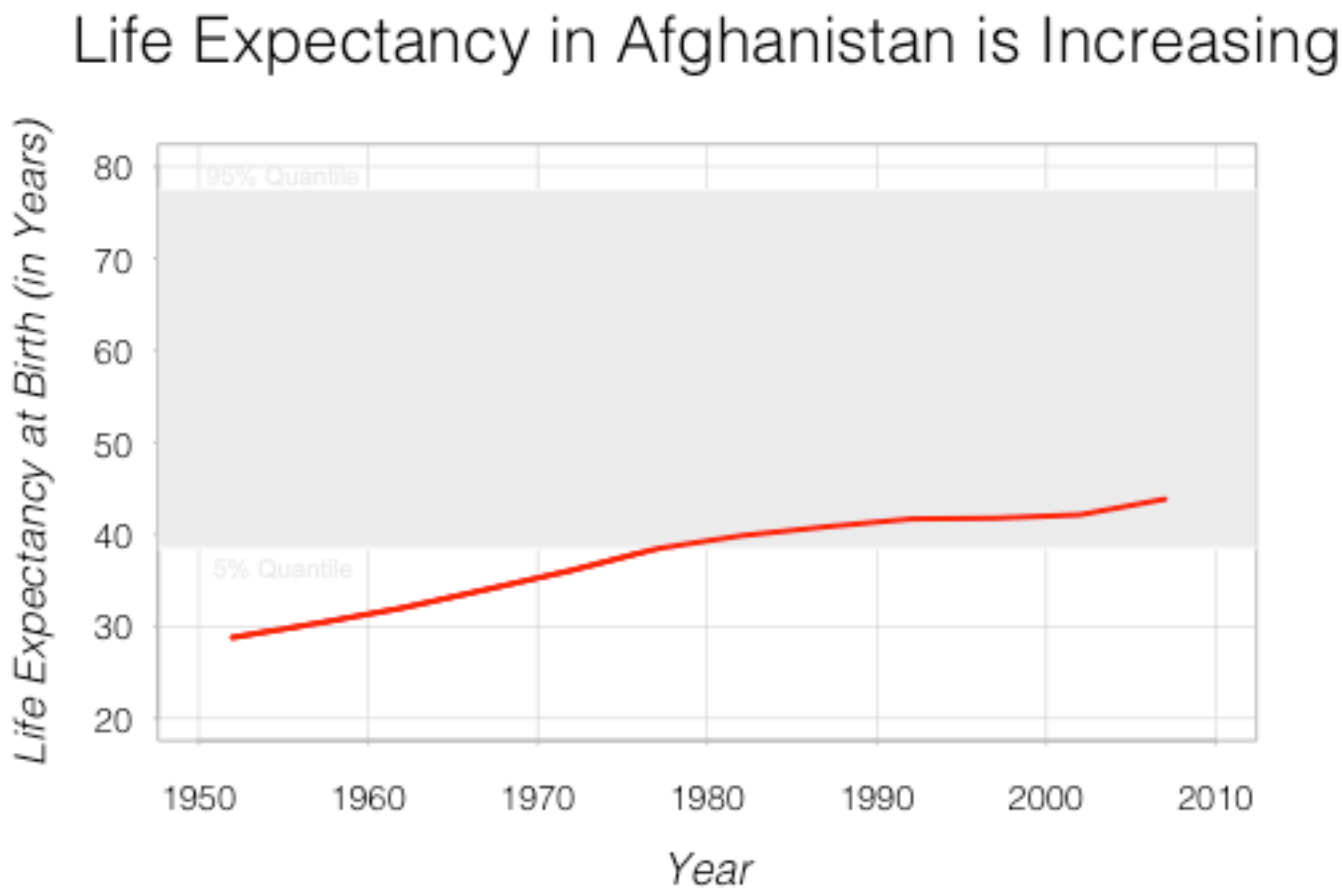
Some Low-level Functions

```
normal.life <- quantile(data$lifeExp, c(.05, .95))
rect(1940, normal.life[1], 2020, normal.life[2], col="grey92",
border=F)
```

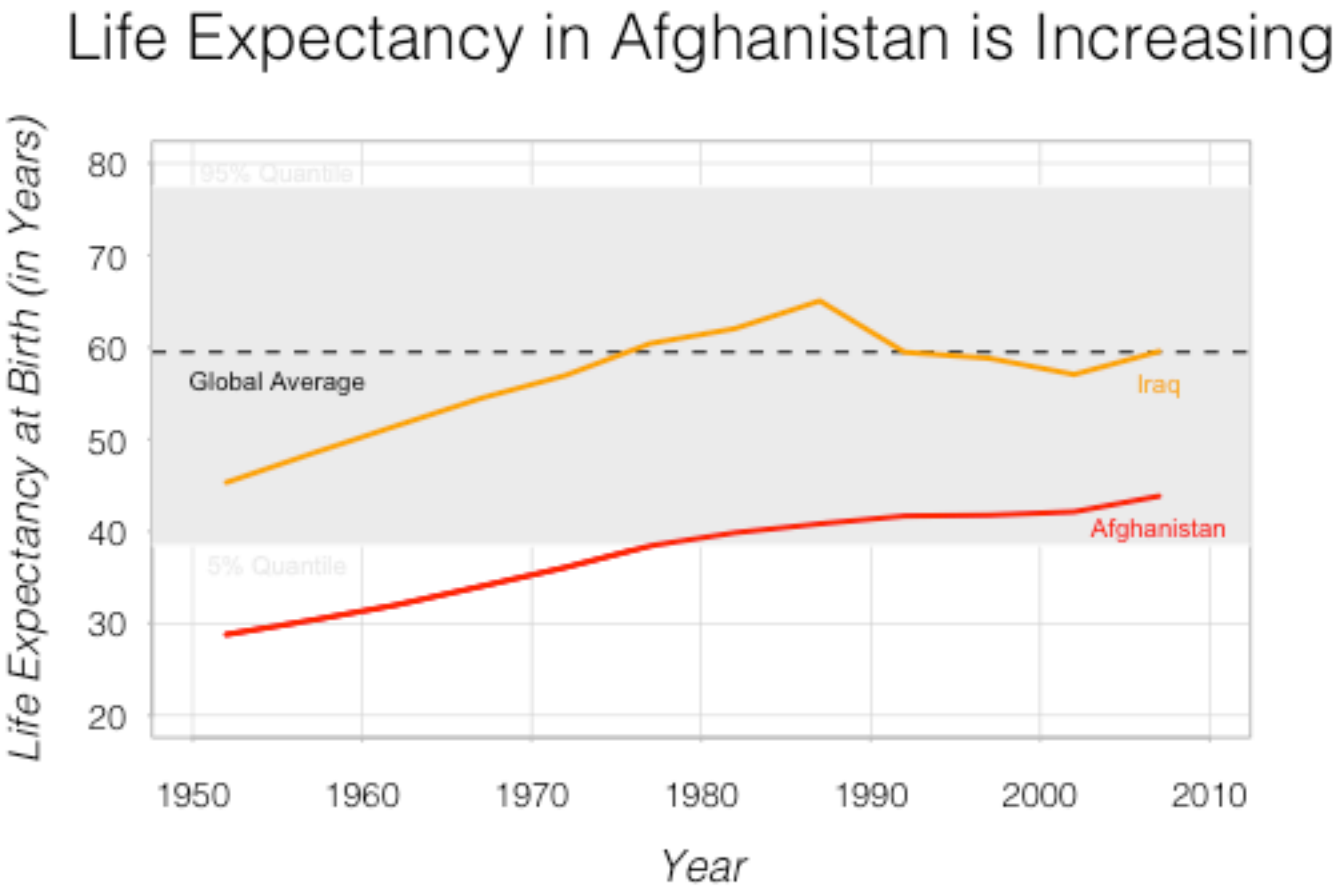


Some Low-level Functions

```
lines(afg$year, afg$lifeExp, lwd=2, col="red")
```



Some Low-level Functions

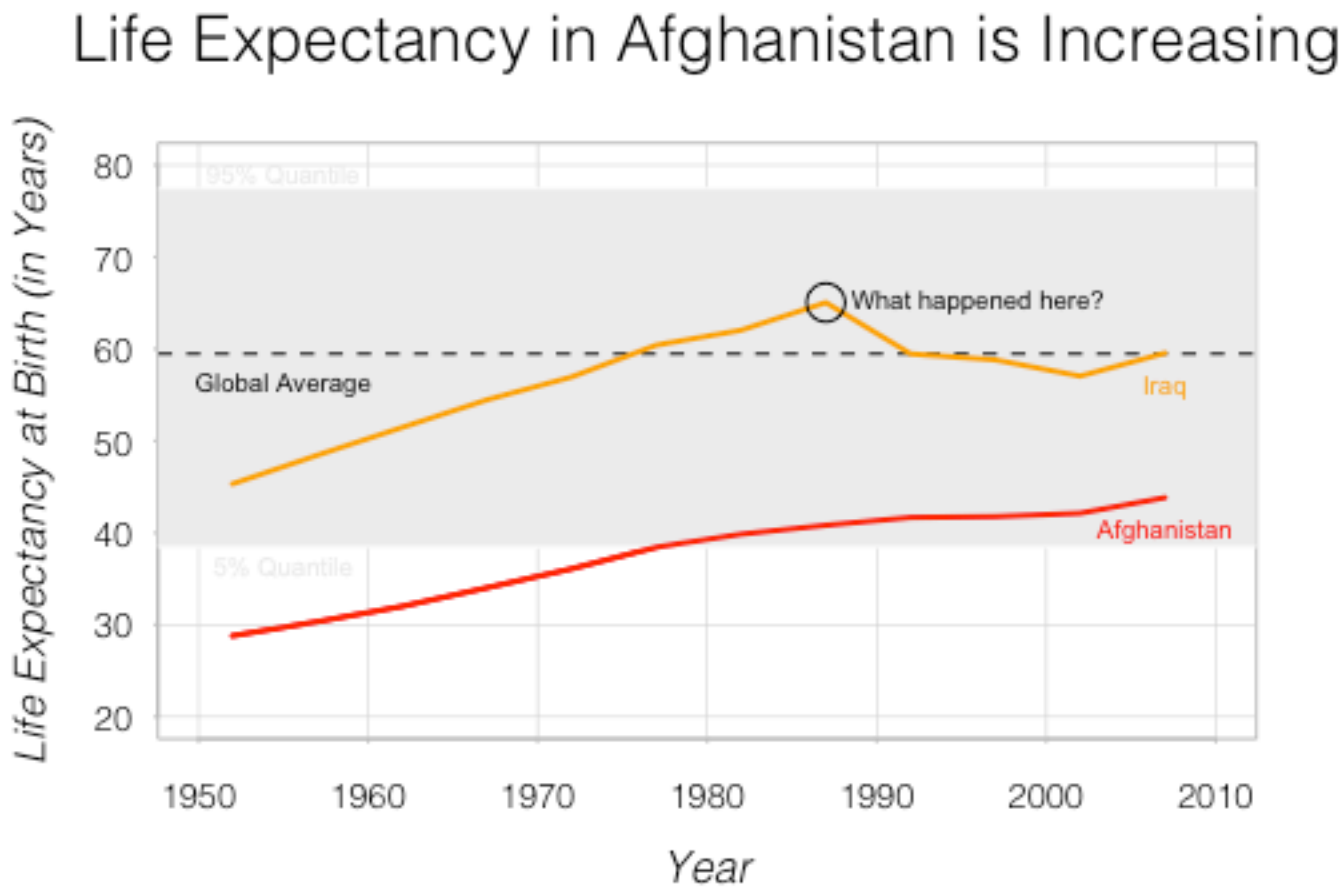


Some Low-level Functions

```
iraq <- data[data$country=="Iraq",]

lines(iraq$year, iraq$lifeExp, lwd=2, col="orange")


text(iraq$year[12], iraq$lifeExp[12], "Iraq", pos=1,
col="orange", cex=.6)

text(afg$year[12], afg$lifeExp[12], "Afghanistan", pos=1,
col="red", cex=.6)
```

Some Low-level Functions

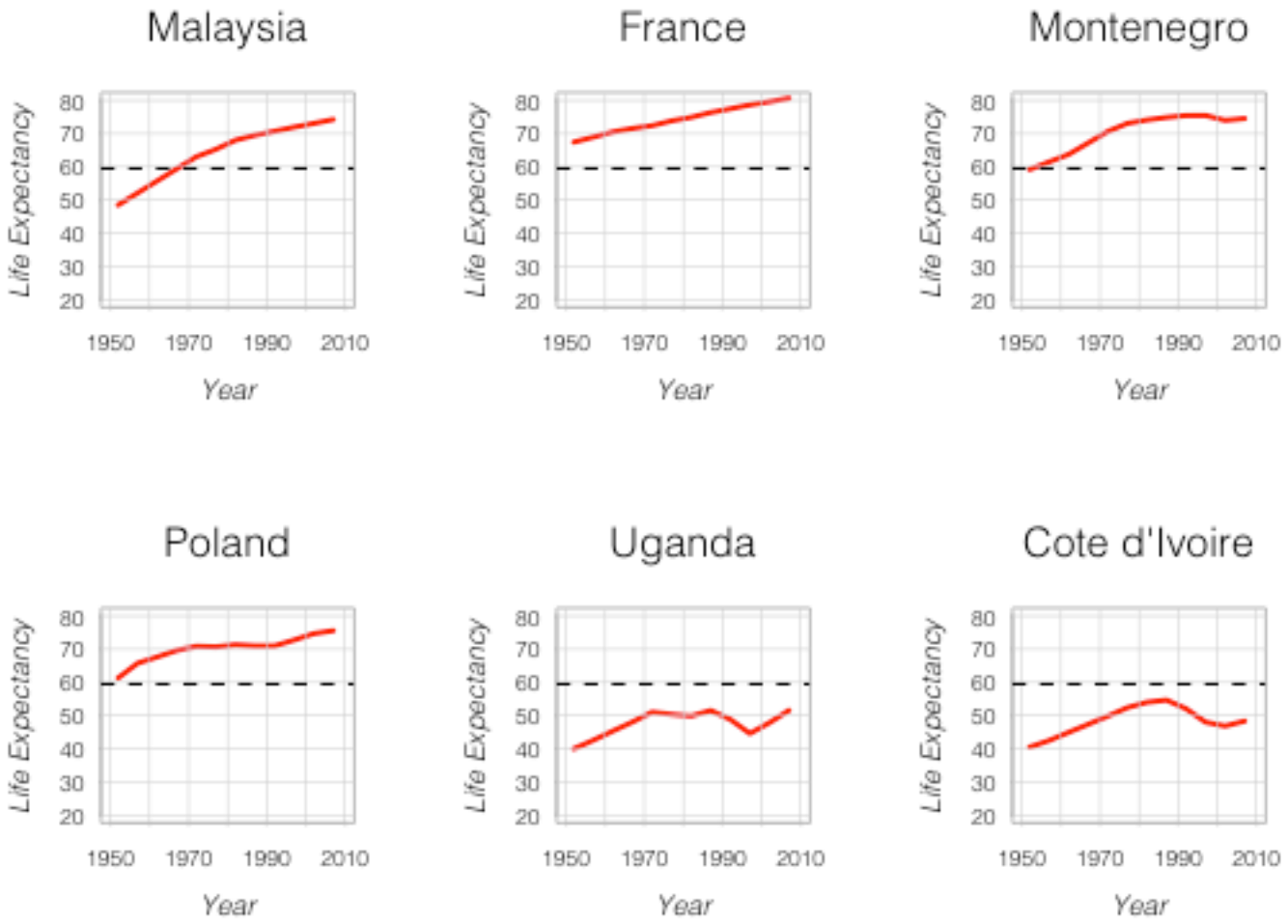
```
points(iraq$year[8], iraq$lifeExp[8], pch=21, cex=2,
col="black")
```



Arranging Multiple Plots

Arranging Multiple Plots

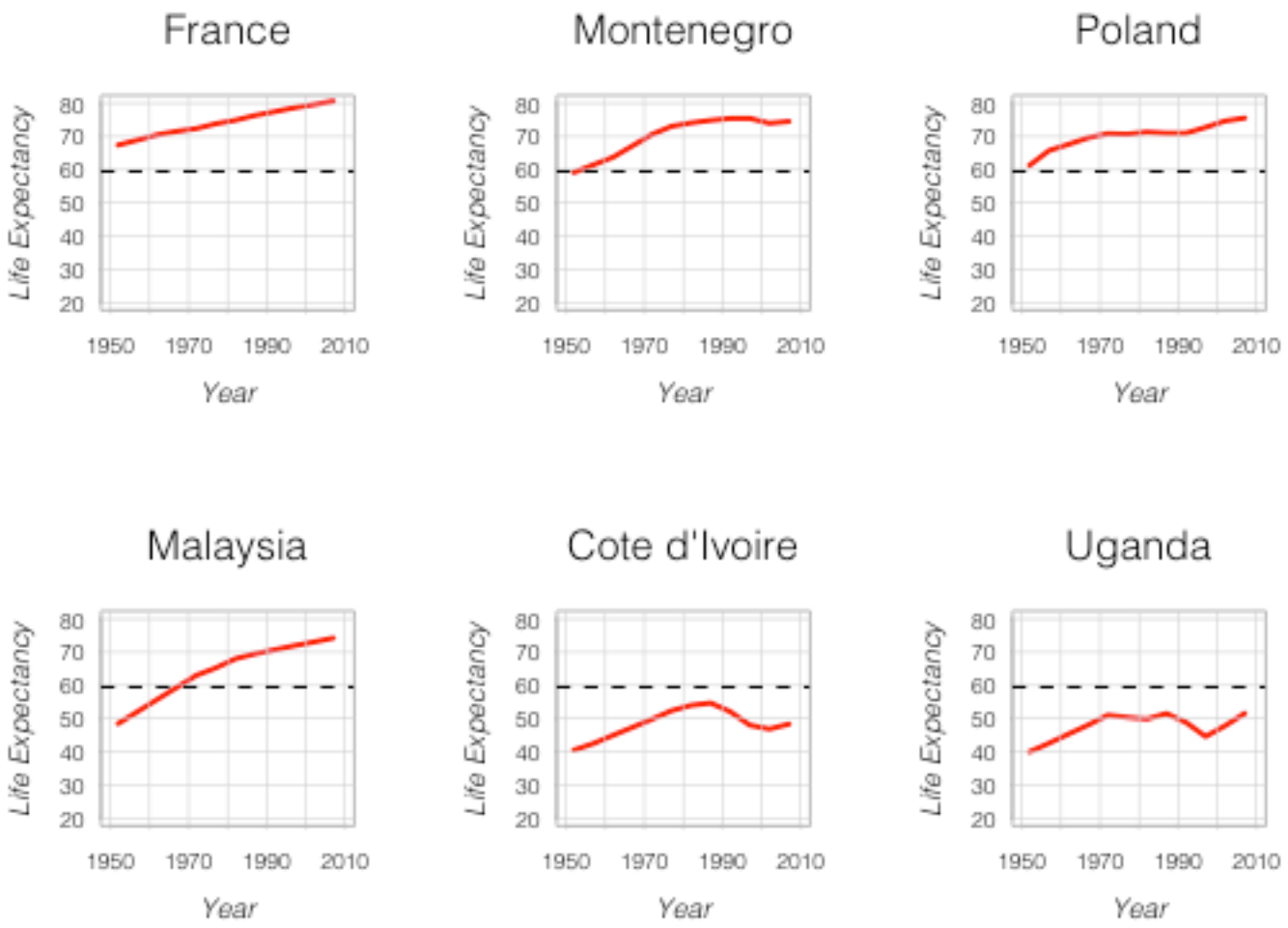
```
par(mfrow=c(2, 3))
```



Arranging Multiple Plots

```
country.samp <- sample(unique(data$country), 6)


for(i in 1:6){


  plot(data$year[data$country==country.samp[i]],
data$lifeExp[data$country==country.samp[i]], type="l", lwd=2, col="red",
     main=country.samp[i],
     xlab="Year", ylab="Life Expectancy",
     ylim=c(20, 80), xlim=c(1950, 2010),
     family="Helvetica Light", font.main=1, font.lab=3,
     cex.main=1.5, cex.lab=1, cex.axis=.8)


  grid(lty=1, lwd=.5)
  abline(h=average.life, lty=2, col="black")
}
```

Arranging Multiple Plots



Arranging Multiple Plots

```
means <- rep(NA, 6)

for(i in 1:6){
  means[i] <- mean(data$lifeExp[data$country==country.samp[i]])
}

ord <- order(means, decreasing=T)

> ord
[1] 2 3 4 1 6

for(i in ord){
  plot(…)
}
```